

OSDU SPI

The Complete Guide: From an Upstream Commit to a Running
Azure Platform

Source material compiled from the *Azure/osdu-spi* and *Azure/osdu-spi-stack* repositories

September 2026

Contents

About This Document	4
Prologue: The Problem	6
What OSDU Is, and What a Provider Is	6
Why a Permanent Fork	8
Two Repositories, One Answer	10
Book One: Keeping the Code Alive	12
The Shape of the Fork	13
Becoming a Fork	16
Generate the Branch, Do Not Merge Into It	19
The Cadence	23
Building Someone Else's Code	28
Trust, and the Token Problem	31
The Second Tier	34
Book Two: Somewhere to Run	38
Four Owners, Four Boundaries	39
The Azure-Only Bet	41
Letting Azure Run the Cluster	43
Bicep, and Why the CLI Stays Thin	46
Ordering Is the Design	48
Identity, Which Is Two Different Problems	52
Getting In	55
Making an Empty OSDU Useful	57
Fifty Minutes, Start to Finish	60
Book Three: The Handshake	63
Two Systems Meet	64
Trust Before Anything Runs	67
The Descriptor	70
The Borrow	72
Proving It	76
Giving It Back	79
What the Seam Reveals	81

Epilogue	83
The Things That Are Not True	83
Writing Down What Was Learned.....	87
Glossary	90
Appendix A: The Engineering System Register	94
Appendix B: The Platform Register	97
Appendix C: Operational Reference	100
Sources	103

About This Document

This is a single, self-contained explanation of how Azure builds and runs OSDU.

It covers two repositories that are usually documented separately. One of them, `osdu-spi`, is an engineering system: it maintains Azure's forks of the OSDU community services, keeps them current with an upstream project that never stops moving, and produces container images. The other, `osdu-spi-stack`, is a deployment platform: it provisions Azure infrastructure and brings up a complete, running OSDU environment on it.

Reading about either one alone leaves a gap. The engineering system produces images but has nowhere to run them. The platform runs images but does not say where a trustworthy one comes from. Between them is a handshake — a fork borrows a slot in a shared environment, proves its candidate image against it, and gives the slot back — that neither repository's documentation describes in full, because each owns only one side of it.

This document follows the whole path instead. It is organized as three books:

Book One — Keeping the Code Alive explains the engineering system: how a permanent fork stays synchronized with a community project that is deleting the very code the fork depends on.

Book Two — Somewhere to Run explains the platform: how a resource group becomes a running OSDU with working identity, ingress, and data.

Book Three — The Handshake explains the seam, in order, from both sides. This is where the book is going, and the first two books are written to make it legible.

A prologue introduces the problem both systems answer. An epilogue collects what is openly admitted not to work, and what the projects wrote down after learning it the hard way.

The reader is assumed to be technically literate but is not assumed to know OSDU, Kubernetes, GitOps, Maven, or Azure's identity model. Each is introduced where it first matters.

The material is drawn from the projects' own documentation — architecture overviews, subsystem design guides, runbooks, and two registers holding seventy-two architectural decision records between them — and checked against the workflows, scripts, and infrastructure code that actually implement it. Where this document cites an exact branch name, file path, command, exit code, or duration, that string was read from the source rather than from prose about the source. The decision records are unusually valuable because they preserve not only what was chosen but what was rejected, and why. Where a trade-off is described here, the trade-off is the project's own stated reasoning.

A note on honesty. Both projects document their own failures with unusual directness. One records a feature that crashed on every single invocation for months while reporting success. The other records

a teardown command whose green result does not prove anything was deleted. Both list gates that are designed, tracked, and not yet built. Those admissions are preserved here rather than smoothed over, because they are frequently the most instructive parts of the story.

Prologue: The Problem

What OSDU Is, and What a Provider Is

The translation problem

The energy industry has a data problem that is fundamentally a translation problem.

A single operator might hold seismic surveys, well logs, drilling records, production measurements, and geological interpretations, gathered over decades, stored in dozens of incompatible systems, produced by competing vendors who each had their own idea of what a “well” is. Merging two companies means merging two of these archives. Adopting a new analysis tool means writing yet another adapter.

OSDU — the Open Subsurface Data Universe — is the industry’s collective answer. Developed under the OSDU Forum, it is both an open standard and an open-source reference implementation defining a common data platform for subsurface and energy data. It specifies a set of services with agreed-upon APIs: a storage service holding records, a schema service defining what those records may look like, a search and indexer pair making them findable, a legal service tracking compliance tags, an entitlements service answering authorization questions, a partition service keeping tenants separate, and others for files, workflows, and unit conversion.

The reference implementation lives in GitLab, at the Open Group’s community instance. It is large, it is actively developed by many contributors across several companies, and it never stops moving.

The provider model, and the seam called SPI

OSDU was designed to run on any cloud. That ambition creates a structural requirement.

Every one of those services needs somewhere to actually store things. Records need a document database, entitlements needs a graph, files need object storage, search needs a search engine, and inter-service notification needs a message bus. OSDU’s answer is a provider model: the business logic of each service is shared, and the cloud-specific plumbing is pushed behind swappable implementations.

In the OSDU source tree this appears as a directory convention. A service named `partition` has a `partition-core` module holding the shared logic, and then `provider/partition-azure`, `provider/partition-aws`, `provider/partition-gc`, and others — one set of rules, several sets of plumbing. The interface between them is a *service provider interface*, and that seam is what gives both Azure projects their name.

It is worth pausing on what this means for a cloud vendor. Implementing a provider is not a matter of configuration. It is writing real code — the Azure implementation of every storage, messaging, and identity operation each service needs — and keeping that code compiling and passing tests against shared logic that other people are changing daily.

Three things called SPI

The abbreviation is used for three different things in this ecosystem, and confusing them makes everything harder to follow. Because this book covers two of the three in full, they are distinguished here once, and the distinction is used consistently from this point forward.

The interface. The original meaning: OSDU’s own pluggability seam, the boundary between shared service logic and a cloud-specific implementation. This is a property of OSDU’s source code and belongs to the community. Neither Azure project owns it.

The Stack. `osdu-spi-stack`, an Azure project that provisions Azure infrastructure and deploys a complete running OSDU environment onto it. It answers the question “how do I get a working OSDU on Azure.” Book Two is about the Stack.

The engineering system. `osdu-spi`, an Azure project that answers a different question: “where does the Azure provider code come from, and how does it stay current with a community project that is still moving.” It produces the service repositories and the container images that the Stack deploys. Book One is about the engineering system.

Where the meaning is not obvious from context, this book names which one it means.

Why a Permanent Fork

A fork cannot be a snapshot

Azure needs its own provider implementation, developed on its own schedule, held to its own review and security standards, and released as artifacts Azure can support. That requires a repository Azure controls. The obvious mechanism is a fork.

The difficulty is that a fork is usually understood as a snapshot: you copy a codebase, you change it, and the two drift apart. That model fails here in both directions.

If the fork stops taking upstream changes, it stops being OSDU. Compatibility with the community standard is the entire value proposition; a divergent implementation that no longer tracks the specification is a proprietary archive with extra steps.

If the fork takes upstream changes by hand, the cost compounds. The project's own framing names four failure modes. Integration is slow, and slowest precisely when an upstream change touches an interface the Azure implementation depends on. Local modifications drift, so each synchronization is harder than the last. In a shared tree a build failure in any one provider's implementation blocks every other provider. And without deliberate tracking, nobody can say which upstream release a given fork version actually contains.

The system's own comparison is blunt about the delta it targets:

Aspect	Manual fork management	This system
Synchronization	Weekly or monthly, by hand	Daily, as a reviewable pull request
Conflict resolution	Wherever the merge happened	Isolated in a dedicated branch
Release coordination	Tracked by hand, if at all	Correlation tags against upstream releases
Integration testing	After conflicts are resolved	At each branch stage

The fork, in other words, must be permanent and continuously integrated. Not a copy. A relationship.

The deletion that forces the design

There is one more constraint, and it is the one that gives the whole architecture its particular shape.

Upstream intends to remove its Azure provider implementations.

This inverts the usual fork problem. Normally a fork worries about upstream changing code the fork depends on. Here the fork must contend with upstream *deleting* code the fork depends on, and

deleting it permanently. After that deletion, `provider/partition-azure` will not exist upstream at all.

The consequence is that the fork cannot merely track upstream. It must take permanent ownership of a specific subtree, seeded once from the newest upstream revision that still contains it, and then maintain that subtree itself, forever, while continuing to receive everything else from upstream on a daily cadence.

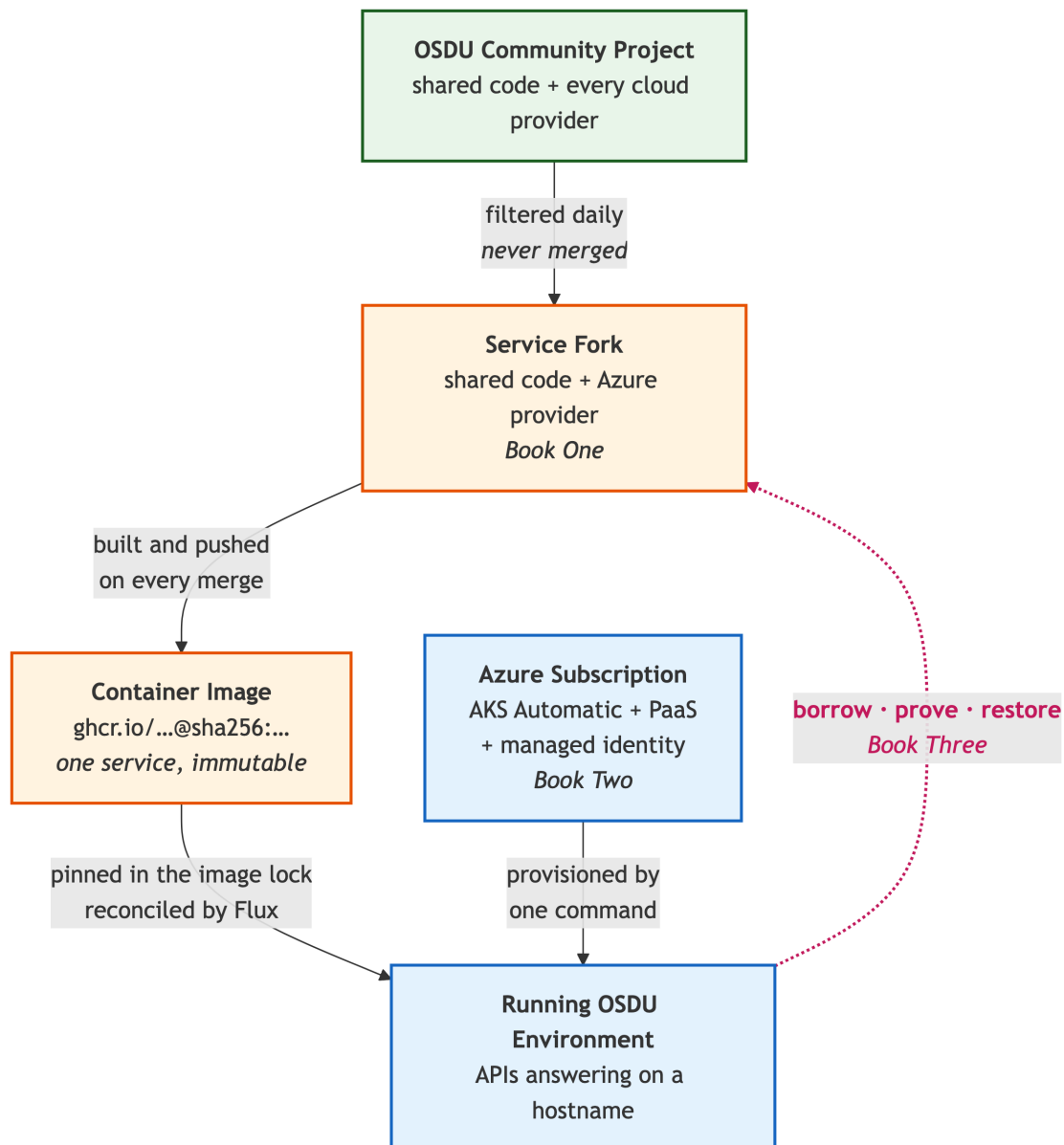
Ownership therefore does not run around the repository boundary. It runs *through the tree*. Some directories in a service fork are upstream-owned and regenerated continuously. Other directories in the same repository, at the same commit, are fork-owned and must never be touched by synchronization. Book One is largely about how that line is drawn and enforced.

The deletion also explains why a deployment platform is needed at all, and why it is a sibling project rather than a feature of the fork. Once Azure owns the provider code, Azure owns proving it works. Proving that a provider implementation works is not a unit-test question. It requires a real Cosmos DB, a real Service Bus, a real identity system, and the other nine services running alongside it. Somebody has to be able to produce that environment on demand.

Two Repositories, One Answer

The shape of the whole system

Put the two constraints together — a permanent fork that owns provider code, and a need to prove that code against real Azure — and the two-repository split follows.



The whole path. The community project supplies shared code; the engineering system keeps an Azure provider alive alongside it and publishes images; the platform turns Azure plus those images into a running environment. The dashed return path is the handshake of Book Three: the fork borrows the environment to prove an image before releasing it.

`osdu-spi` is a GitHub *template repository*. A service fork created from it inherits a complete set of workflows, composite actions, configuration schemas, rulesets, and build assets. The template owns the logic; each fork owns its data. The system manages eight core OSDU services — partition, entitlements, legal, schema, file, storage, indexer, and search — with the explicit intent that adding a ninth adds no new patterns. Every fork runs the same workflows, and a fix made once in the template arrives everywhere as a reviewable pull request.

`osdu-spi-stack` is a Python CLI, a set of Bicep templates, and a Git repository of Kubernetes manifests. One command turns an empty Azure subscription into a running OSDU: an AKS cluster, five Azure data services, three in-cluster middleware systems, ten OSDU services, working ingress with TLS, and enough seeded data — partitions, entitlements, and roughly 1,386 schemas — that the APIs answer real requests.

They meet at exactly one seam. The engineering system builds a candidate image, borrows a slot in a running Stack environment, pins its image there temporarily, runs its acceptance suites against the live APIs, and restores what was there before. Everywhere else the two are independent: the Stack can deploy community images with no fork involved, and the engineering system's build and synchronization machinery runs with no Azure environment in sight.

What these systems are not

Neither project claims more than it does, and the disclaimers are load-bearing.

The engineering system is **not a deployment system**. It does not provision Azure resources, does not run a Kubernetes cluster, and does not operate a live OSDU environment. It borrows one, briefly, to prove an image.

It is **not a general-purpose fork tool**. The filter engine that makes it work is specific to the shape of OSDU service repositories: a Maven multi-module tree with a provider directory convention, testing modules, and a core module.

The platform is **not a production configuration**. That sentence appears verbatim in its own documentation. It is a development and testing system: single-region, no backup or disaster-recovery design, secrets and scaling tuned for iteration rather than for operating a business on it.

It is **not portable**. Every data service with a managed Azure equivalent uses that equivalent. Moving the Stack to another cloud is a fork, not a flag.

And neither is finished. Both maintain public lists of things that are designed, tracked, and not yet built. Book Three reaches a seam whose core is shipped and whose remaining gates are openly listed as unbuilt, and the epilogue collects the rest.

Book One: Keeping the Code Alive

The engineering system: `osdu-spi`

Everything in this book follows from one sentence in the prologue: upstream is going to delete the Azure provider code, and Azure has to keep it alive anyway.

That single fact rules out the ordinary answers. A fork that merges from upstream will eventually be asked to merge a deletion. A fork that stops merging stops being OSDU. What is needed instead is a repository where some directories are continuously regenerated from a moving upstream, other directories at the very same commit are owned permanently by Azure, and no automated process is ever allowed to confuse the two.

The seven chapters here build that repository up in order. The first two describe its shape and how one comes into existence. The third is the mechanism the whole system rests on, and is the longest for that reason: the decision to stop merging from upstream entirely and generate the upstream branch instead. The remaining four cover the daily rhythm that keeps the fork current, what it takes to build somebody else's Java, how the system decides which automation may hold credentials, and a second tier of forks operated by customers.

One thing is deliberately deferred. These chapters produce a container image and stop short of proving it works, because proving it requires a running Azure environment — the subject of Book Two, and of the handshake in Book Three.

The Shape of the Fork

One Template, Many Forks

Every service fork needs the same workflows and the same actions, but a different upstream URL, a different service name, different Maven coordinates, and different repository settings. The template repository pattern separates those two things: the template owns the logic, and each fork owns its data in configuration files.

The propagation mechanism is a workflow called `Sync Template`, which runs in every fork daily at 08:00 UTC. It compares the fork's copy of the engineering system against the template's current state, and when the template has moved it opens a pull request in the fork. The fork owner chooses when to merge it.

This is the payoff the pattern exists for. Fixing a bug in `sync.yml` once is enough; the fix arrives in every repository as a reviewable pull request, rather than as a hand edit repeated eight times with a typo in the sixth. The propagation is asynchronous and review-based rather than immediate, which is the accepted cost.

Two Contexts in One Repository

The template repository contains two directories of workflows, and the distinction between them is load-bearing.

`.github/workflows/` is the **template's own** continuous integration. It runs only in `osdu-spi` itself: the template's development CI, its release management, its documentation build, its CodeQL analysis, its OpenSSF Scorecard run, and the two initialization workflows that configure a new fork.

`.github/template-workflows/` is **what forks receive**. At initialization these files are deployed into the fork as its `.github/workflows/`. This directory holds synchronization, cascade, cascade monitoring, validation, build, release, template sync, adoption, settings application, registry retention, integration cleanup, Dependabot validation, and the fork's own CodeQL configuration.

Before the split existed, every fork carried template-only infrastructure it never used. Separating the two is an instance of a rule the project states explicitly and applies repeatedly: *split what fails differently*. Template CI failing and fork initialization failing are unrelated events, and code that handles both in one file has two unrelated failure modes in one place.

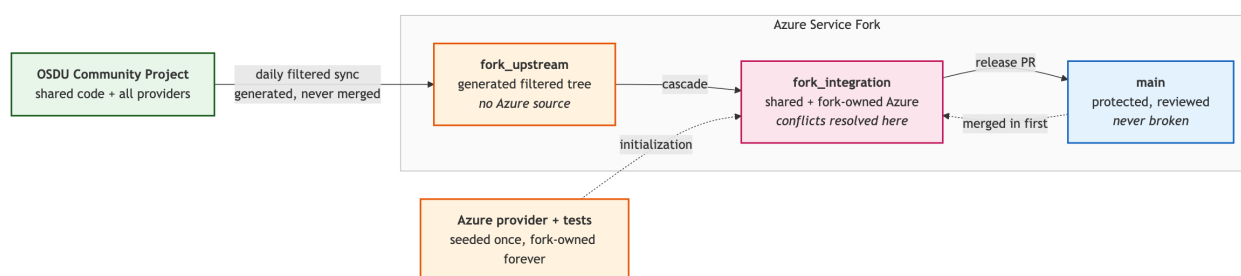
Three Branches, Three Different Jobs

A service fork maintains three long-lived branches. Their names are exact, and they are not interchangeable.

fork_upstream holds the upstream-derived tree. After the change described shortly it is not a mirror of upstream but a *generated* tree: shared upstream code with the non-Azure providers and upstream deployment machinery removed, and references to the fork-owned Azure modules injected. It is written only by synchronization and read only by the cascade. Nothing else touches it.

fork_integration is the workspace. It is where the generated upstream tree is combined with the fork-owned Azure source, where conflicts are resolved, where Maven versions are stamped, and where the build and tests run. It is allowed to break.

main is production. It is protected: pull requests only, one approving review, and two required status checks named exactly **CodeQL** and **Validation Summary**. It receives only changes that have already been integrated and validated somewhere else.



The three-branch flow. Upstream change enters a generated branch, is integrated and validated in a workspace branch, and reaches production only as a reviewed release pull request.

Why three rather than two, or one? The register considered all three options. A two-branch strategy is simpler. Merging upstream directly into the working branch is simpler still. Both were rejected for the same reason: they put conflict resolution and build failure on the branch the team is trying to ship from.

The project's own account of the payoff is worth quoting in substance. When integration breaks, the break is contained on `fork_integration` and fixed there, while feature work on `main` continues. Adding a stage did not slow anyone down. It removed the case where one upstream change stopped everybody at once.

The branches also differ in what is allowed to happen to them. `fork_upstream` accumulates: an unmerged synchronization pull request is not a problem, it is a queue. `fork_integration` is disposable and is reset to `main` after a release lands. `main` is never permitted to be broken.

Protection is applied from checked-in configuration. Two ruleset files, `default-branch.json` and `integration-branch.json`, are reconciled onto the repository weekly by a `Settings Apply` workflow. The `default-branch` ruleset requires review and status checks on `main`. The `integration-branch` ruleset protects `fork_upstream` and `fork_integration` from deletion only — automation must remain able to force-push them.

Ownership Runs Through the Tree

The single most important structural fact about a service fork is that ownership is not uniform across the repository.

Upstream-owned, and regenerated on every synchronization:

- the service’s core module and shared code
- the community acceptance test module
- the shared build configuration those modules inherit

Fork-owned, and never written by synchronization:

- `provider/<svc>-azure` — the Azure provider implementation
- `testing/<svc>-test-azure` — the Azure integration test source
- `.github/` — the engineering system itself
- `build/` — the canonical Dockerfile and entrypoint
- `.spi/` — the acceptance descriptor that Book Three takes up in detail

A single commit on `main` therefore contains code with two different provenances and two different maintenance contracts. The synchronization mechanism must respect that line precisely, on every run, forever. The next chapter is about how.

Two Tiers

The model extends one level further than the eight Microsoft-maintained service repositories.

A customer organization can run a true GitHub fork of a service repository through the same machinery, in a different mode: instead of filtering an upstream community project, their fork mirrors the service repository’s finished `main` verbatim. They build and release with their own credentials, against their own environment, and features they prove can return as contribution pull requests through the GitHub fork network.

The two tiers run the same workflows, the same three branches, and the same cascade. They differ only in what their upstream is and how synchronization treats what it receives. A later chapter describes the second tier in full; it is mentioned here because several mechanisms in the intervening parts carry a “unless this is a mirror” clause, and that clause is otherwise inexplicable.

Becoming a Fork

Initialization Is a Conversation

A new service fork begins as a GitHub repository created from the template. The first push triggers a workflow named `Initialize Fork`, which detects that it is running in a fork rather than in the template, creates the system labels, and opens an issue titled **Repository Initialization Required**.

That issue is the entire user interface. A human replies to it with the upstream repository to track. A second workflow, `Initialize Complete`, triggers on the comment and does the actual work: it validates that the upstream repository exists and is reachable, records it in the `UPSTREAM_REPO_URL` variable, generates the filtered `fork_upstream` branch, creates `fork_integration`, seeds the fork-owned Azure trees, plants a filter configuration file, deploys the template workflows into the fork, applies repository settings and rulesets, and finally sets `INITIALIZATION_COMPLETE` to true.

The ordering of that last step matters. `INITIALIZATION_COMPLETE` is written last, so a run that fails halfway leaves the repository visibly uninitialized rather than half-armed. Several workflows check the flag before doing anything, which means a partially configured fork does not start synchronizing against an upstream it has not finished validating.

Re-running after a failure converges. Steps that already completed detect their own output and skip or reconcile, so recovery is a re-run rather than a cleanup.

Split What Fails Differently

Initialization is two workflows rather than one because the original single workflow failed in two unrelated ways.

Handling user input — parsing an issue comment, validating a URL, dealing with a person who pasted the wrong thing — is a different problem from setting up a repository, with different error conditions and different recovery. Combining them produced a file where a malformed comment and a GitHub API permission error arrived at the same error handler.

The split gives each half one responsibility and one failure mode. It is the same rule that separated the template's own workflows from the workflows forks receive, applied at a smaller scale.

The Bootstrap Problem

There is a circularity in a self-configuring template, and the project solves it with a directory.

Some logic must run in the fork's very first commit, before any synchronization could possibly deliver a fix to it. If that logic lives in the normal shared action directory and it has a bug, the bug

cannot be repaired through the ordinary propagation path, because the ordinary propagation path is the thing that is broken.

The system therefore maintains three distinct homes for executable logic, separated by when they must be available:

- `.github/actions/` holds composite actions that are synchronized to forks and updated through template sync. Most logic lives here.
- `.github/local-actions/` holds logic that must exist in a fork's first commit, before any sync could deliver a fix. It also holds the test harnesses.
- `.github/scripts/` holds durable helper scripts.

This is downstream of a more general rule about shell code. Bash embedded in workflow YAML cannot be run locally, duplicates across files, and is hard to debug — and the project had already discovered the same script existing in two places. Substantial shell now lives in a `.sh` file wrapped by a composite `action.yml`, so the same file runs from a workflow, from a developer's shell, and from a test harness. Each script carries a shebang, `set -euo pipefail`, and writes its results both to `$GITHUB_OUTPUT` and to standard output so it is debuggable outside Actions.

The extraction is deliberately uneven, and the register says so: initialization helpers and state management are extracted, while orchestration remains inline. One extracted action, `issue-state-manager`, currently has no workflow calling it.

The One-Time Seed

Initialization performs an operation that happens exactly once in the life of a repository and can never be repeated.

Because upstream intends to delete its Azure provider implementations, the fork must capture them first. Initialization seeds `provider/<svc>-azure` and `testing/<svc>-test-azure` from the newest upstream revision that still contains them. From that moment, those trees are fork property. Synchronization will never write to them again, and late upstream fixes to Azure source — if any arrive before the deletion — require a deliberate manual port.

Initialization is also the only place in the system where a textual merge across unrelated histories occurs. The generated upstream tree and the seeded Azure tree have no common ancestor, so joining them once requires an actual merge. Everywhere else, as the next chapter explains, the system stopped merging on purpose. The register notes this exception precisely: the `-X theirs` merge survives only in initialization, for the first join of unrelated histories.

Adoption, the Other Door

A second entry path exists for the customer tier.

A customer organization does not instantiate the template. They create a real GitHub fork of a service repository — which, being a fork, already inherits every deployed workflow from its parent's `main` — and then run a workflow called `Adopt Fork` manually.

Adoption detects the fork's parent as its upstream, creates `fork_upstream` and `fork_integration` at the merge base, creates the labels, sets `UPSTREAM_REPO_URL`, sets `SYNC_MODE` to `mirror`, applies the rulesets, enables the workflows that apply to a mirror while leaving template sync and Scorecard disabled, and sets `INITIALIZATION_COMPLETE` last.

The reason customer repositories must be forks rather than template instances is a GitHub constraint with an architectural consequence, and it is explained in the chapter on the second tier.

Settings as Code

Repository configuration is not clicked into a settings page. It is reconciled.

The rulesets live as JSON under `.github/rulesets/`, and a `Settings Apply` workflow re-applies them weekly, every Monday at 04:00 UTC. This means a well-meaning manual change to branch protection is reverted within a week, and it means a new required check can be rolled to every fork by editing a file in the template.

Initialization also configures the security posture: CodeQL analysis, Dependabot with a fork-appropriate configuration, and the rulesets. A fork is secured at creation rather than secured later by an audit.

Generate the Branch, Do Not Merge Into It

This part describes the mechanism that makes the rest of the system possible. It is also the clearest example of the project abandoning an obvious approach after the obvious approach failed repeatedly in a specific, diagnosable way.

The Conflict That Kept Coming Back

The naive plan is straightforward and almost works.

Take upstream. Delete the provider directories the fork does not ship — `provider/partition-aws`, `provider/partition-gc`, the GitLab CI configuration, the upstream deployment machinery. Commit the deletions. Then, every day, merge the new upstream into your branch with a strategy option that prefers upstream's version of any conflicting file.

This fails, and it fails in a way that gets worse rather than better.

`git merge -X theirs` resolves *content* conflicts by preferring the other side. It does not resolve **modify/delete** conflicts at all. When upstream modifies `provider/partition-aws/pom.xml` and the fork has deleted it, Git does not know whether the answer is the modification or the deletion. It stops and asks.

Worse, the conflict is not a one-time cost that decays. It recurs on *every* synchronization, because as the merge base advances, each new upstream change to a deleted file produces the same unanswerable question again. And the natural human recovery — `git add -A` to accept the working tree and move on — silently restores the deleted files, quietly undoing the strip.

The project's summary of this is one sentence: *stripping the other cloud providers from a fork cannot be done by deleting them and merging upstream afterwards.*

The Reframe

The insight that resolves it is a change of category rather than a better merge strategy.

`fork_upstream` is written only by synchronization and read only by the cascade. No human commits to it. No feature branches off it. It has exactly one producer and one consumer.

A branch with those properties does not need to be *merged into*. It can be **generated**. If the content of `fork_upstream` is a pure function of the upstream tip and a filter configuration, then synchronization can compute the correct tree from scratch every time and write it directly. Nothing is textually merged, so the modify/delete conflict class cannot occur — not “is handled,” cannot occur.

The project states the rule as a heading: *generate the branch; do not merge into it.*

Building a Tree with Plumbing

The implementation uses Git's low-level commands rather than its porcelain, because the goal is to construct a specific tree object rather than to reconcile two working trees.

Synchronization reads the upstream default branch into a scratch index, checks that index out into a generation directory, runs the filter engine over the directory, and then writes the result as a commit with two parents:

```
GIT_INDEX_FILE=$SCRATCH git read-tree upstream/$DEFAULT_BRANCH
GIT_INDEX_FILE=$SCRATCH git checkout-index -a -f --prefix=$GEN/
upstream-filter --config .github/upstream-filter.yml --checkout $GEN
git commit-tree $TREE -p fork_upstream -p $UPSTREAM_SHA
```

The last line is the subtle one. The new commit's tree is the filtered content, but its parents are the previous `fork_upstream` tip *and* the upstream commit it was derived from. The result is merge-shaped: `git log` shows a normal history, upstream commits are reachable, `git blame` works, and the relationship to upstream is recorded in the commit graph. But no merge algorithm ever ran. The shape is a record of provenance, not the output of a reconciliation.

What the Filter Removes

The filter's configuration lives in each fork at `.github/upstream-filter.yml`, and the engine classifies every top-level entry, every testing module, every POM profile, and every FOSSA module declaration.

Removed from the generated tree:

- `provider/` — every provider except Azure
- `devops/` — upstream's deployment machinery, including `devops/azure`, because the Stack owns its own charts
- non-Azure `testing/` modules
- `.gitlab-ci.yml` — upstream's pipeline definition
- `<svc>-core-plus` — a module with no consumers after the strip

Kept, and continuously updated: the core module, the community acceptance test module, and the shared build configuration.

Injected: references to the fork-owned Azure modules, so the generated tree's build wiring points at code that lives only on `fork_integration` and `main`.

That last point deserves emphasis. The generated branch contains *references* to Azure modules but not the Azure modules themselves. `fork_upstream` is provider-less by construction. This is why validation builds it with a Maven profile of `core` alone, a detail the chapter on building returns to.

The rejected alternatives are recorded with their genuine advantages. Sparse checkout or a custom merge driver would be less invasive but leave unwanted paths in the tree. Stripping `testing/` wholesale would produce a smaller tree but lose the Azure integration tests. Keeping `devops/azure` would preserve upstream’s pipelines, but the Stack does not use them. Emitting the Azure POMs into the generated tree would simplify version stamping, but would reintroduce exactly the modify/delete conflicts the design exists to eliminate.

Halt on the Unknown

The filter classifies everything. When it encounters something the configuration does not name — a new top-level directory, an unrecognized testing module, a POM profile nobody declared — it does not guess.

It exits with code 2, and the run stops. An issue is opened carrying the labels `sync-failed` and `human-required`, naming what it could not classify.

This is a deliberate and somewhat expensive choice. The cheap alternative is a default: keep unknown things, or drop them. Both are wrong in a way that is invisible. Defaulting to *drop* means a new shared upstream module is silently deleted from the fork and nobody notices until a build fails for an unrelated-looking reason. Defaulting to *keep* means upstream’s new AWS provider quietly lands in an Azure fork.

The project generalizes the rule across two components — the filter and the acceptance resolver of Book Three both refuse unknown inputs — and states the reasoning in one line: *guessing would have been convenient exactly once and wrong forever after*.

Provenance in Two Trailers

Every generated commit carries two Git trailers.

`Upstream-Sha` records the exact upstream commit the tree was derived from. This is what makes “which upstream state is this” answerable without inference.

`Filter-Rev` records the filter’s identity: the engine version joined to the first twelve hex characters of a SHA-256 over the configuration file’s bytes. Two commits with the same `Upstream-Sha` and the same `Filter-Rev` are guaranteed to have the same tree. Two commits with the same `Upstream-Sha` and *different* `Filter-Rev` values are the same upstream state seen through different rules.

That second case is the one the trailer exists for. Because the generated tree is a function of both the upstream tip and the configuration, a change to the configuration alone changes the correct output even when upstream has not moved. The duplicate-suppression logic described in the next chapter depends on this: its cache key hashes the configuration file, the filter engine, and the generation script together, so editing the filter is recognized as a change rather than dismissed as a no-op.

Proving the Filter Before Trusting It

Generating a tree instead of merging one trades a loud failure for a quiet one. A merge conflict stops the world and demands attention. A filter bug produces a perfectly clean commit containing the wrong tree.

The register names this as the cost of the approach, and the response is verification built into the engine, which runs in four modes: `generate`, `verify`, `stamp`, and `seed`.

Verification is of three kinds. **Negative** checks assert that things which should have been removed are absent. **Positive** checks — configured as `expected_kept` — assert that things which must survive actually did, which is the check that catches an over-aggressive rule. And a **non-mutating re-run** compares the engine's output against the current tree to raise an alarm when a new upstream module appears, before it can be silently mishandled.

There is also a manual escape hatch, used during the partition service's cutover: if the fork-owned trees are ever damaged, they can be restored from the previous commit directly, because they are ordinary tracked files.

```
git checkout HEAD -- provider/partition-azure testing/partition-test-azure
```

What the Fork Now Owns Forever

The consequence of this design is stated plainly in the register, including the part that hurts.

The Azure trees stop receiving upstream changes. If upstream fixes a bug in its Azure provider before deleting it, that fix does not arrive; somebody must port it deliberately. The cascade takes on responsibility for version wiring, because the fork-owned POMs must be kept consistent with upstream-derived coordinates that now move independently of them. And a bug in the filter manifests as a wrong tree rather than as a conflict, which is why the verification modes exist.

What is bought is a synchronization that cannot produce the recurring conflict class, runs unattended daily, and yields a reviewable pull request rather than a merge session.

The Cadence

Parts III and IV described a fork's creation and the mechanism that keeps its upstream branch correct. This part describes what actually happens, day after day, once the machinery is running.

A Sync That Remembers

Every day at midnight UTC, the `Sync Upstream` workflow runs in each fork. It fetches the upstream tip, generates the filtered tree as described above, and — if anything changed — opens a pull request into `fork_upstream` from a branch named `sync/upstream-YYYYMMDD-HHMMSS`.

The pull request is titled  `Sync with upstream <version>` and carries the commit list in its body, capped at 200 entries and 60,000 characters. A companion tracking issue titled  `Upstream Sync Ready for Review` is opened with the labels `upstream-sync` and `human-required`.

The interesting engineering here is not the synchronization. It is the memory.

A scheduled workflow runs on a fresh, stateless runner. It has no recollection of the previous day. A synchronization that cannot remember what it already did opens a duplicate pull request every single day that a human takes to review the first one — which is how the system originally behaved, producing notification fatigue, branch clutter, and genuine confusion about which pull request was current.

The obvious places to keep state were each rejected for a concrete reason. Git notes bring their own merge conflicts. Git config does not survive between hosted runner invocations. External storage adds a dependency and a failure mode. Closing the old pull request and opening a new one destroys the continuity of any review already in progress. Leaving both open is the problem itself.

The chosen answer is that durable state lives in GitHub's own metadata, split across three places by lifetime:

- A **hidden marker** in the tracking issue body, `<!-- upstream-sha: ... -->`, records the upstream SHA that the currently open synchronization is for. It lives exactly as long as the open synchronization does.
- The **`upstream-sync` label** is how the workflow finds its own branches, pull requests, and issues without guessing at names.
- A **repository variable**, `SYNC_LAST_EVALUATED_SHA`, records the last upstream state that was evaluated and found to produce no change in the fork's tree. Its value is not a bare SHA but `<upstream sha>:<generation revision>`, so a change to the filter configuration invalidates it even when upstream has not moved.

From those three, the daily decision is a four-case table:

Existing sync PR	Upstream changed	Action
No	Yes	Create the branch, the pull request, and the tracking issue
Yes	No	Leave everything untouched
Yes	Yes	Force-push the existing branch; update the title and body
No	No	Do nothing

Abandoned branches — more than 24 hours old with no associated pull request — are cleaned up.

The error handling is worth noting because it follows the same philosophy as the filter. A GitHub API read failure **fails the workflow** rather than proceeding on an assumption; guessing that no pull request exists when the API merely timed out would create the duplicate the mechanism exists to prevent. A missing or malformed marker compares as *changed*, which is the safe direction, and is repaired on the next write. A failed variable write costs one repeated evaluation, not a failed synchronization.

The whole mechanism is tested outside GitHub Actions, by a shell harness that exercises malformed lookups, abandoned-branch cleanup, legacy and current and CRLF-corrupted markers, malformed SHAs, stale generation revisions, unreadable variables, and workflow ordering.

The Meta Commit

Release Please, the tool that computes versions and changelogs, requires Conventional Commits — messages shaped like `feat:` or `fix:` or `chore:`. Upstream OSDU does not write them, and never will.

The obvious fixes all involve rewriting history: squash the upstream range into one properly formatted commit, or rebase and reword. Both destroy the upstream history, and with it `git blame` and any ability to bisect a regression to an upstream change.

Instead, synchronization *adds* one synthetic commit summarizing the range, and derives its type by a fixed rule over the commits it contains:

breaking beats `feat`, `feat` beats `fix`, and anything unclassifiable counts as `fix`.

A rule rather than a judgment. The same input always produces the same version bump, the full upstream history remains intact underneath, and versioning stays automatic.

The project frames the general lesson as *add metadata instead of rewriting history*.

The Cascade

Merging the synchronization pull request into `fork_upstream` is a human decision. What happens afterward is the cascade, and it is where integration actually occurs.

The `Cascade Integration` workflow merges in a specific order. First it merges `main` into `fork_integration`, bringing the workspace up to date with whatever production already contains. Only then does it merge `fork_upstream` on top. Reversing this would integrate upstream against a stale baseline and produce conflicts that are artifacts of the ordering rather than real disagreements.

It then asserts that the fork-owned paths are intact — the Azure provider and test trees must be present and unmodified by the merge — stamps the fork-owned POM versions to match the upstream-derived coordinates, and runs the full build and test.

On success it opens a pull request into `main` from a temporary branch named `release/upstream-YYYYMMDD-HHMMSS`, labeled `validated`. On conflict it opens an issue titled  `Cascade Conflicts: Manual Resolution Required` and stops. On a build or test failure it opens  `Integration Validation Failed: Build/Test Errors` and stops.

In both failure cases the work is now sitting on `fork_integration`, isolated, where a human can fix it without touching `main` and without blocking anyone's feature work.

The review is deliberately **asymmetric**. Not every hop requires the same scrutiny. Merging a generated upstream tree into an integration workspace is a low-stakes, reversible act; promoting a validated result to production is not. Making every stage symmetric would be simpler to describe and slower and noisier to live with.

Labels Are the State Machine

A cascade spans hours or days and crosses many stateless workflow runs. It needs somewhere to keep its state.

The first attempt used issue assignment — assign the issue to a person, and the presence of an assignee means somebody owns it. This failed on organization accounts, on renamed users, and on ordinary API errors.

Labels have none of those problems and are queryable from any workflow without a lookup. So labels became the state machine, not merely decoration:

`cascade-active` means a cascade is in flight. `cascade-blocked` means it stopped and needs help. `cascade-ready`, `cascade-failed`, and `cascade-escalated` mark the remaining positions. `validated` marks a result that passed integration. `upstream-held` marks work queued behind an active cascade. `integration-blocked` marks a stalled integration. And `human-required` means exactly what it says.

That last label carries a second, less obvious function: **its removal is a signal**. When a human fixes a conflict and deletes the `human-required` label, that deletion is what tells the system it may try again.

The tracking issue, meanwhile, is not just a notification. It is the audit trail for a process spanning days, and it is where the hidden state marker lives.

Human-Triggered Steps Need a Sweeper

The cascade is meant to be started by a person after reviewing the synchronization pull request. People forget. Transient failures leave work stuck halfway.

`Cascade Monitor` runs every six hours. It compares `fork_upstream` against `fork_integration`, finds merged synchronization pull requests with no corresponding cascade and starts one, retries cascades whose `human-required` label has been removed, escalates conflicts that have been stale for 48 hours, and reports overall cascade health.

The project is careful about what this is: *a safety net behind the manual path, not a replacement for it*. The sweeper does not make the cascade automatic. It makes forgetting non-fatal.

Cleaning Up After Itself

When a release pull request merges into `main`, an `Integration Branch Cleanup` workflow resets `fork_integration` back to `main`.

It triggers on a merged pull request to `main` that either carries the `upstream-sync` label, came from `fork_integration`, or came from a branch beginning with `release/upstream-`. It also closes any `upstream-held` issues, releasing work that was queued behind the cascade that just finished.

This is what makes `fork_integration` genuinely disposable. It never accumulates history of its own, so it never becomes a third thing to maintain.

Releases, and the Tag That Says Which Upstream

`Release Management` runs on every push to `main`, except when the push touched only `.github/**`.

`Release Please` computes the next semantic version from the conventional commits since the last release — including the meta commit that synchronization contributed — and opens or updates a release pull request containing the version bump and the generated changelog. Merging that pull request publishes the release.

Then two things happen that are specific to a fork.

First, a **correlation tag** is created, formatted as `<release-tag>-upstream-<upstream-version>`. This is the answer to a question a fork must be able to answer and a normal repository never faces: *which upstream release does this fork release contain?* Without it, that question is archaeology.

Second, the container image is **retagged, not rebuilt**. Validation already built and pushed an image identified by its commit SHA. The release adds the semantic-version tag to that existing image. The bits that were tested are the bits that ship, because they are literally the same bits. The workflow polls for the source image — up to 60 attempts at 20-second intervals, roughly twenty minutes — rather than assuming it is already there. The acceptance test image is tagged alongside it, which is what makes “run the tests that shipped with release X” a durable operation months later.

The release workflow has no manual trigger. A release is a consequence of merging to `main`, not an action someone takes.

Building Someone Else's Code

Java 17, Maven, and a Repository You Do Not Own

OSDU services are Java and Maven projects, so the build path is fixed: Java 17 from Temurin, Maven, and JaCoCo 0.8.11 for coverage. The logic is implemented once as composite actions — `java-build` and `java-build-status` — and every fork calls the same ones.

The build depends on a Maven repository the project does not control. OSDU's shared artifacts are published to the Open Group's community GitLab instance, and the fork's Maven settings point at it:

```
https://community.opengroup.org/api/v4/groups/17/-/packages/maven
```

Access is anonymous, which removes a credential but not a dependency: if that host is unreachable, every build in every fork fails. The register lists this consequence without mitigating it.

Two other consequences are accepted explicitly. Java is pinned at 17 inside the action, so a fork needing a different runtime needs a different workflow. And a fork that is not a Java and Maven project is out of scope — Gradle and Bazel were both considered and rejected on the grounds that upstream is Maven and conversion work buys nothing.

Fast feedback for developers is a separate workflow. `Build` and `Test` runs on pushes to ordinary branches only, explicitly excluding `main`, `fork_integration`, `fork_upstream`, `sync/**`, and `dependabot/**`, so that feature work gets a quick build and coverage report without colliding with the branches the automation owns. Its artifacts are retained for two days.

The Profile Trap

This is a small detail with an outsized failure mode, and it is a good illustration of why the engineering system centralizes build invocation instead of letting each fork improvise.

Maven profiles can be marked `activeByDefault`. OSDU service POMs use this for the core module. The Azure fork only needs the core and Azure profiles, so the apparently correct command is `-P azure`.

It is wrong. **Maven deactivates every `activeByDefault` profile the moment any explicit `-P` is passed.** Building with `-P azure` therefore silently drops `core`, and the build fails to resolve `<svc>-core` — a confusing error a long way from its cause.

The rule adopted is consequently phrased in terms of what is never emitted: the build always passes a non-empty `-P`, and the default is `core,azure` rather than `azure`. A repository variable, `MAVEN_PROFILE`, overrides it for forks whose layout is unusual, and two such forks exist:

`entitlements`, whose provider sits at `provider/entitlements-v2-azure`, and `indexer-queue`, which carries provider modules in its default `<modules>` block.

There is one context-sensitive exception. Because `fork_upstream` is provider-less by construction, building it with `core,azure` would fail — there is no Azure source there to build. Validation therefore selects the profile from the branch and the tier:

```
maven_profile: ${ ( (github.base_ref == 'fork_upstream' || github.ref_name ==  
'fork_upstream')  
    && vars.SYNC_MODE != 'mirror')  
    && 'core' || vars.MAVEN_PROFILE || 'core,azure' }}
```

Read plainly: on `fork_upstream` in filter mode, build `core` only; otherwise use the fork's override if it has one, and `core,azure` if it does not. The `SYNC_MODE` clause is there because at the mirror tier `fork_upstream` *does* carry provider code, for reasons the chapter on the second tier explains.

The accepted cost is that CI produces no signal at all about non-Azure providers. That is intentional — building four providers to learn about one wastes runner time — but it means an upstream change that breaks AWS is invisible here.

The Dockerfile Nobody Maintained

The original design assumed each service repository carried a usable Dockerfile. That assumption was false, and the counter-example is preserved in the register because it is instructive.

`provider/partition-azure/Dockerfile`, inherited from upstream, specified a base image of `openjdk:8-jdk-alpine` for a Java 17 service, copied a JAR named `partition-aks-1.0.0.jar` that no longer existed, and assumed a module-relative build context that CI does not use. It was not subtly wrong. It had not worked in a long time, and nobody had noticed, because nothing built it.

The ruling is that the engineering system owns one canonical `build/Dockerfile` and CI uses that one regardless of what a fork happens to carry. The image is built on `mcr.microsoft.com/openjdk/jdk:17-azurelinux`, pinned by digest. The Dockerfile itself is deliberately minimal — an `ARG JAR_FILE` and a `COPY` — and the knowledge of *which* JAR is supplied by the build machinery, defaulting to:

```
provider/<SERVICE_NAME>-azure/target/*-spring-boot.jar
```

with `SERVICE_TARGET_JAR` available to disambiguate when a module produces more than one. Application Insights instrumentation is added by `ADD --checksum` into `/opt/agents/` and attached at runtime by `build/docker-entrypoint.sh` only when `APPLICATIONINSIGHTS_CONNECTION_STRING` is present, so the agent is inert by default.

The image is built from the JAR that Maven already produced, rather than compiled inside the Dockerfile. Building inside would be more self-contained, but it duplicates the Maven build, discards the dependency cache, loses the coverage report, and is slower. The consequences accepted are that forks cannot easily diverge their Dockerfile, that stale in-repository Dockerfiles still exist but are unused, and that ARM64 validation on the push path pays for QEMU emulation on an amd64 runner.

A Public Registry, and a Setting That Cannot Be Fixed

Service images go to GitHub Container Registry, publicly.

The reasoning is narrow and situational. These images are test artifacts destined for a shared AKS cluster run by the Stack. Pushing from GitHub Actions with the built-in `GITHUB_TOKEN` requires no extra credential, and public visibility means the cluster pulls anonymously with no `imagePullSecret` to provision and no chart wiring to maintain. Azure Container Registry remains a viable fallback and was rejected as unnecessary rather than unsuitable.

The failure mode is specific and worth knowing. If a package's visibility flips to private, every cluster pull fails with `ErrImagePull`. The `Settings Apply` workflow checks visibility and reports it — but **cannot fix it**, because GHCR exposes no API for changing package visibility. The system can tell you the thing is broken and cannot repair it, which is a rare and honest thing for a register to record.

Images accumulate, so a `GHCR Retention` workflow runs weekly, keeping the newest five snapshot tags per branch and deleting SHA-tagged images older than thirty days.

Coverage Without a Threshold

JaCoCo runs and publishes a report. No threshold is enforced.

The register states this as an accepted consequence rather than an oversight: coverage is informational. For a fork whose majority of code is written by somebody else and regenerated continuously, a coverage gate would measure upstream's habits more than the fork's, and would fail builds for reasons no local change caused.

Trust, and the Token Problem

Automation that can write to repositories, publish container images, and authenticate to Azure is an attack surface. This part covers how the system decides which code is allowed to run with which credentials.

Why the Default Token Is Not Enough

GitHub Actions provides every run with a `GITHUB_TOKEN`. Under Microsoft's organization policy that token is read-only, and the permission to create pull requests is disabled outright. Personal access token lifetimes are also constrained by policy.

This is not an inconvenience to be worked around; it eliminates the two obvious approaches. A synchronization workflow that cannot open a pull request cannot do its job, and a personal access token belonging to an individual is both against policy and a single point of failure tied to one person's employment.

The system uses **GitHub App installation tokens**. An organization-owned App is installed on selected repositories, and workflows needing elevated permissions mint a token at the start of the run using a pinned action:

```
actions/create-github-app-token@bcd2ba49218906704ab6c1aa796996da409d3eb1 # v3.2.0
actions/create-github-app-token@67018539274d69449ef7c02e8e71183d1719ab42 # v2.1.4
```

Two versions are in use: the newer in `settings-apply.yml` and `init-complete.yml`, the older in several other template workflows. Actions are pinned by commit SHA rather than by tag throughout, because a tag can be moved and a SHA cannot.

The credentials are two secrets, `RELEASE_APP_ID` and `RELEASE_APP_PRIVATE_KEY`, which must exist wherever a token is minted. The App needs read and write on contents, pull requests, administration, workflows, and variables — administration because it applies rulesets, workflows because initialization deploys workflow files.

The costs are real and recorded. Tokens expire after one hour, so a long-running job must be structured around that. Organization admin approval is required to install the App, and changing its permissions requires re-approval. A helper script, `rotate-app-key.sh`, exists because private keys need rotating.

Know the Event Context Before Building a Workaround

Cascade triggering was unreliable for weeks, and the cause was not a bug in the cascade.

GitHub's `pull_request` event runs the workflow file **as it exists on the pull request's branch**. When the cascade is triggered by a pull request whose branch does not yet contain the current workflow definition, the workflow that runs is the wrong one, or none at all.

`pull_request_target` runs the workflow from the **target** branch instead. Switching the trigger fixed weeks of unreliability with one line.

The project draws the general lesson as a heading — *know the event context before building a workaround* — because the alternative path was to build increasingly elaborate machinery around a misunderstanding of the platform.

That same context has a sharp edge, and the system ran into it. A job that checks out the pull request's head under `pull_request_target` is running untrusted code in a privileged context, and CodeQL flagged it as a cache-poisoning vulnerability. The response was structural: the validate-only Docker job **never sets a checkout ref** and never takes that lane at all.

The Guard Clause Is the Policy

Two jobs in the system hold real credentials. `docker-push` has `packages: write` on the registry. `deploy-test`, the subject of Book Three, holds an Azure federated identity.

The rule is that credential-bearing jobs run only in trusted contexts, and the trust boundary is drawn at *who can put code in the branch being built*.

Excluded, always: `pull_request_target` runs, pull requests from external forks, Dependabot pull requests, and ordinary `workflow_dispatch` invocations. Allowed: pushes to `main` and `fork_integration`, cascade pushes to `fork_integration`, and pull requests whose head repository is the same as their base repository. Pushes to `fork_upstream` may take the non-credentialed build path only.

In practice the boundary is an `if:` clause carrying those conditions, repeated verbatim on each sensitive job. That repetition is deliberate, and the project defends it in one line: **the clause is the policy; a reviewer noticing a missing guard is not**. Any future credential-bearing job copies it.

The honest cost is that external contributors get no image push and no deployment signal on their pull requests. A maintainer-approved override input exists in the schema but is explicitly *not yet wired*, and the register says so rather than implying the capability exists.

A Required Check That Always Runs

A required status check has a name, and GitHub matches it literally. This creates a class of problem that has nothing to do with security and everything to do with mechanics.

The ruleset on `main` required a check named `CodeQL`. The CodeQL workflow's jobs had different names, and it skipped analysis entirely on configuration-only pull requests — correctly, since there

was no code to analyze. A skipped job does not report a check. The pull request waited forever for a check that would never arrive.

Compounding it, the ruleset also used a `code_scanning` rule with an alert threshold, which waits for SARIF results. For a documentation-only change, no SARIF is ever produced, so the pull request blocked permanently.

The fix has two halves. The workflow gained a summary job named exactly `CodeQL` that runs with `if: always()` and reports the aggregate outcome by inspecting the results of the jobs before it — so the check always exists, whether analysis ran or was correctly skipped. And the `code_scanning` rule was removed from the ruleset entirely.

The alternative of uploading an empty SARIF file to satisfy the rule was rejected as misleading, which is consistent with the system's general posture: report the true state rather than manufacture a passing signal.

What was genuinely lost is worth stating. Blocking a merge on an alert severity threshold is no longer possible; findings are triaged through the Security tab rather than enforced by the ruleset. The register records that as a cost, not as a non-issue.

Dependabot, Split by Ownership

Dependency scanning follows the same ownership line as everything else.

The **template** scans what the engineering system owns: `.github` for GitHub Actions versions, daily at 08:00, and `/build` for Docker base images, daily at 08:30.

A **fork** scans only its application code — Maven ecosystems across the root, the core module, the acceptance test module, the Azure provider, and the testing modules — daily at 09:00, with updates restricted to **patch level only**. Both configurations cap open pull requests at five and label them `dependencies`.

Patch-only is the interesting restriction. Minor and major upgrades are excluded not because they are unimportant but because they conflict with upstream OSDU's own dependency choices; a fork that unilaterally advances a shared library diverges from the community build. The accepted cost is that minor and major upgrades must be scheduled deliberately rather than arriving automatically, and the register acknowledges the technical debt that implies.

Forks do not scan `.github` at all, because those updates arrive through template sync. During initialization, open Dependabot pull requests outside the fork's Maven ecosystem are closed, because a freshly created fork inherits the template's scanning configuration until its own replaces it — and Dependabot caches ecosystem lists, so the inherited configuration can produce noise before the swap takes effect.

The Second Tier

Why It Must Be a True Fork

A customer organization running OSDU on Azure may need changes: a fix they need before the next release, an integration specific to their business, a feature they would like contributed back.

The mechanism for contributing back imposes the architecture. **A pull request's head branch must live in the target repository's fork network.** A repository created from a template is not in the service repository's fork network, no matter how identical its contents, and therefore can never open a pull request against it.

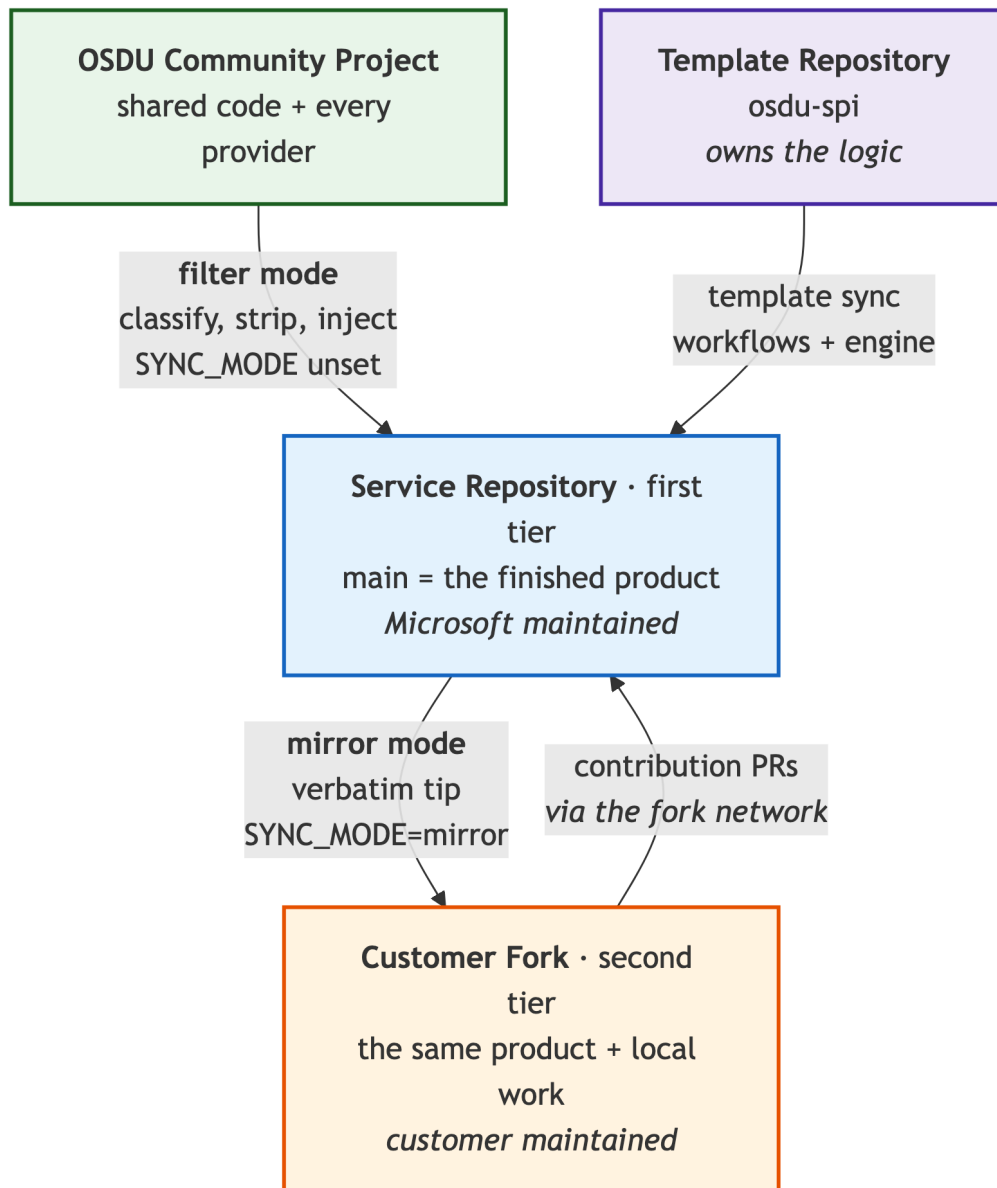
That single GitHub constraint settles the creation mechanism: customer repositories are real forks, never template instantiations. A convenient side effect is that a fork inherits every deployed workflow from its parent's `main` at the moment of creation, which is what makes adoption lightweight — the machinery is already there before the adoption workflow runs.

Mirror Mode

The upstream filter reduces a sprawling community project to the Azure product. Pointed at a service repository's `main`, it would be actively destructive.

That branch **is already the product**. Running the filter against it would halt on `.github/`, on `build/`, and on the changelog, none of which its configuration classifies — and it would strip `provider/`, which is the code a downstream consumer needs most.

So the second tier does not filter. `SYNC_MODE=mirror` takes the upstream tip **verbatim**, through the same commit plumbing described earlier: the same merge-shaped commits, the same `Upstream-Sha` trailer, and a sentinel `Filter-Rev: mirror`. Provenance and no-op semantics are identical at both tiers while the filter engine never runs at all.



The two tiers. A community project is filtered into a service repository; the service repository is mirrored verbatim into a customer fork, which can contribute back through the fork network.

The tier must be selected by a variable because **it cannot be inferred from the tree**. The filter configuration file exists at both tiers — at the customer tier it arrives *through the mirror itself* — so its presence says nothing, and deleting it would create a permanent difference against upstream that every subsequent sync would try to reconcile. One repository variable, set once at adoption, is the whole selector. Unset means existing behavior, so introducing the second tier changed no existing repository.

What Changes, and What Does Not

Both tiers run the same workflows, the same three branches, the same cascade, and the same release flow.

	First tier (service repository)	Customer tier (mirror fork)
Created by	Template instantiation + initialization	GitHub fork + Adopt Fork
Upstream	The upstream community project	The service repository's <code>main</code>
Sync behavior	Filter: classify, strip, inject	Mirror: verbatim tip
<code>SYNC_MODE</code>	Unset (or <code>filter</code>)	<code>mirror</code>
Fork-owned content	Azure provider, Azure tests, <code>.github/</code> , <code>build/</code>	Local feature work only
Cascade path assertion and POM stamping	Active	Off — no ownership split to defend
Template sync	Active	Off — updates arrive through the mirror
Sync PR build profile	<code>core</code> only	Full profile set

Two of those rows are consequences of the same fact. At the first tier `fork_upstream` is provider-less, so it builds `core` alone and the cascade must defend an ownership boundary. At the mirror tier `fork_upstream` carries the complete product including `provider/`, so it builds everything and there is no boundary to defend.

Template synchronization is off at the second tier because it would be redundant and conflicting: workflow updates already arrive as part of the mirrored product.

The Round Trip

The full development loop across both tiers:

1. **Upstream change.** The community project merges something. The first tier's filtered sync brings it in; the cascade integrates and releases it to the service repository's `main`.
2. **Mirror in.** The customer fork's next sync mirrors the new `main` verbatim; their cascade merges it with their local work.
3. **Local feature.** The customer develops and proves a feature in their own fork, with their own build, against their own environment.
4. **Contribute back.** The customer opens a pull request from their fork to the service repository. A maintainer approves the run and sees real build signal — while credential-bearing jobs stay off the external head, exactly as the chapter on trust requires.

5. **Round trip.** The merged feature reaches the service repository's `main` and returns to the customer through their next mirror sync, where the cascade converges their local copy with the incoming change.

What the Second Tier Costs

Three consequences are recorded rather than hidden.

Mirror-tier repositories carry the same release automation, so **release conflicts happen**. `CHANGELOG.md` and `.release-please-manifest.json` are edited on both sides of the mirror, and those conflicts are shallow but real and sometimes need manual resolution. Disabling release automation at the mirror tier was considered and rejected, because it would remove the customer's release autonomy — which is much of the point of having their own fork.

Upstream deletions mirror through silently, by design. Mirror mode is verbatim; it does not second-guess a removal.

And mirror-tier validation **needs build credentials earlier** in the onboarding process than the first tier does, because their `fork_upstream` carries provider code and therefore builds the full profile set from the start.

Book Two: Somewhere to Run

The platform: `osdu-spi-stack`

Book One ended with a container image and an unanswered question: does it work?

That question cannot be answered by a unit test. An OSDU service is almost entirely an integration: the partition service is a thin API over Cosmos DB, the entitlements service is a graph traversal, the indexer is a pipeline from Service Bus into Elasticsearch. Testing whether the Azure provider implementation is correct means running it against a real Cosmos DB, a real Service Bus, real identity, and the other nine services it talks to.

So somebody has to be able to produce a complete, working OSDU on Azure, on demand, from nothing. That is this book.

The scope is one command and roughly fifty minutes. `spi up` turns an empty subscription into an AKS cluster, five Azure data services, three in-cluster middleware systems, ten OSDU services and three reference APIs, ingress with a real TLS certificate, and enough seeded data — partitions, entitlements groups, a legal tag, and about 1,386 schemas — that the APIs answer real requests.

The nine chapters move outward from the substrate: who owns what, what went to Azure and what stayed, how the cluster is run, how Azure resources are declared, how workloads are ordered and then deliberately frozen, how identity works in both directions, how traffic gets in, what has to be created before an empty OSDU is useful, and finally the whole thing as a timeline.

Throughout, one disclaimer from the project's own documentation is worth keeping in view: *This is not a production configuration*. It is a development and testing platform, and several of the decisions below only make sense once that framing is taken seriously.

Four Owners, Four Boundaries

The most useful thing to understand about the platform before any of its parts is that four different things change the system, and they do not coordinate at runtime. The architecture overview names them explicitly, and nearly every confusing behavior later in this book is a consequence of a boundary between two of them.

Owner	What it does	Where it stops
CLI and Bicep	Resource group, AKS, networking, identities, Azure PaaS, initial Kubernetes configuration, seed credentials, Flux activation	Re-run explicitly. Flux does not reconcile Azure infrastructure.
Flux	Apply workload manifests and Helm releases in dependency order	Uses the cached Git revision when source fetching is suspended
Kubernetes controllers and operators	Schedule pods, maintain middleware clusters, issue certificates, distribute CA bundles	Continue running independently of Git polling
Operator	Decide when to fetch Git changes or refresh images; diagnose failures; delete environments	A successful CLI exit is not an API-readiness check

That last cell is the one to remember. `spi up` returning zero means the CLI finished its own work. It does not mean OSDU is answering requests. Flux is still reconciling, operators are still starting middleware, and the schema loader may have another hour of work ahead of it. The gap between “the command succeeded” and “the platform is ready” is real, it is measured in tens of minutes, and the system’s answer to it — a machine-readable readiness contract — turns out to be the hinge on which all of Book Three swings.

Three namespaces, and three more that are not ours

The platform creates and reconciles exactly three namespaces:

- **foundation** holds operators and cluster-wide services: the ECK operator for Elasticsearch, the CloudNativePG operator for PostgreSQL, cert-manager, trust-manager, and — in `dns` ingress mode — ExternalDNS.
- **platform** holds the stateful middleware those operators manage: Elasticsearch, Redis, PostgreSQL, Airflow, and the TLS certificates.
- **osdu** holds the OSDU services themselves, the partition and entitlements initialization Jobs, legal-tag seeding, and the schema loader.

Three further namespaces exist but belong to somebody else. `flux-system` holds the Flux controllers, installed and upgraded by the AKS extension. `aks-istio-system` and `aks-istio-ingress` hold the managed service mesh and its ingress gateway, owned by AKS itself.

An early design considered a fourth namespace of its own, `istio-system`, and rejected it: on AKS Automatic the managed Istio add-on already owns the mesh namespaces, and creating a parallel one would mean running a second mesh alongside the platform's.

A seventh namespace, `osdu-flux`, holds the platform's *own* GitOps objects — its `GitRepository`s, its `Kustomizations`, its `HelmRelease`s, its bootstrap input `ConfigMaps`, and its credential seed. The reason it exists rather than putting those next to the controllers in `flux-system` is a policy the cluster enforces: `aks-managed-protect-system-namespace-objects` blocks writes into `flux-system`. Rather than request a policy exemption, the platform keeps everything it owns in a namespace it is allowed to own.

There is one more asymmetry worth noting now because it explains a later chapter. OSDU pods receive Istio sidecars; the middleware pods in `platform` do not. `platform` is deliberately outside the mesh, because sidecar injection runs an `istio-init` container that requires the `NET_ADMIN` capability, and AKS Automatic's Deployment Safeguards reject it.

The Azure-Only Bet

Five systems that left the cluster

The platform's first decision is also its least reversible: every data service with a managed Azure equivalent uses that equivalent.

Need	Service
Records and metadata	Azure Cosmos DB (SQL API)
Entitlements graph	Azure Cosmos DB (Gremlin API)
Blobs and tables	Azure Storage
Events and notification	Azure Service Bus — 14 topics with 14 subscriptions
Configuration and credentials	Azure Key Vault

Five stateful systems therefore live outside Kubernetes entirely. The cluster holds no database that matters. A pod can be deleted, a node can be replaced, the whole cluster can be torn down and rebuilt, and the records survive because they were never in it.

The alternative considered was running the same workloads in-cluster for portability, and it was rejected on the grounds that portability is not a goal: the Stack exists to run Azure-specific OSDU provider code. The documentation's phrasing is worth keeping, because it closes the question permanently — *a different cloud is a fork, not a flag*.

The three that stayed

Exactly three stateful systems stayed inside the cluster, each for a stated reason.

Elasticsearch, managed by the ECK operator, runs three nodes with 128 GiB of storage. It stayed because Azure AI Search is not a drop-in replacement for the Elasticsearch APIs that OSDU's search and indexer services actually use. This is not a preference; it is an API-compatibility fact, and it is the single clearest case where the managed-service-first rule could not be applied.

Redis, from the Bitnami chart, runs one master and two replicas with 8 GiB. **PostgreSQL**, managed by CloudNativePG, runs three instances with 10 GiB of data and 4 GiB of write-ahead log, and exists only to back Airflow. Both had managed equivalents available — Azure Cache for Redis and Azure Database for PostgreSQL — and both were rejected on latency and cost for workloads that are co-located with their only consumers.

Airflow itself runs in `platform` too, but is not stateful; its state is the PostgreSQL cluster beside it. Altogether the in-cluster middleware claims roughly 450 GiB of Premium SSD.

Backup and disaster recovery for any of this is explicitly out of scope, which is consistent with the development-and-testing framing and would not be defensible without it.

Turning the keys off

The most consequential security decision in the platform is a single Bicep property, set in three places and never made configurable.

`disableLocalAuth: true` is hardcoded on the Cosmos DB Gremlin account, on every per-partition Cosmos DB SQL account, and on every per-partition Service Bus namespace. Azure Storage has shared-key access disabled. The effect is that the primary keys and shared-access signatures those services would normally issue simply do not work. There is no fallback path, because the fallback path was deliberately removed.

What replaces them is Entra ID role assignments: Cosmos SQL and Gremlin data contributor roles, Service Bus data sender and receiver roles, granted to a managed identity. No `listKeys()` call appears anywhere in the templates, and the documentation states the invariant directly: *No `listKeys()`-derived credentials are written*. Where a configuration value would normally hold a connection string — the `graph-db-primary-key` entry, for instance — the platform writes the literal string `DISABLED`.

Two alternatives were rejected: running both paths at once, with RBAC available but keys still valid, which leaves the weaker path open and therefore in use; and staying with keys and requesting policy exemptions, which inverts the intended default.

This decision has a cost, and the project states it plainly. Community OSDU images that build a connection string and authenticate with a key do not work against an account whose local auth is disabled. They fail, correctly and permanently, until an image capable of Workload Identity replaces them.

That sentence is where Book One and Book Two touch for the first time. The `indexer-queue` service is the live example: its community image constructs a Service Bus connection string, finds `DISABLED` where a key should be, and cannot authenticate. Fixing it means changing provider code, which means owning provider code, which is the entire premise of the engineering system. The platform's security posture is one of the concrete forces that makes the fork necessary.

Letting Azure Run the Cluster

AKS Automatic, and the version floor

The cluster is created with `sku.name: Automatic`. AKS Automatic is an opinionated mode in which Azure operates much of what a team would otherwise assemble: node provisioning through managed Karpenter, a managed Istio service mesh, Deployment Safeguards as an admission layer, the Key Vault CSI driver, Cilium networking in overlay mode, Managed Prometheus, and Container Insights.

The alternative was standard AKS with a self-managed Istio installation, rejected because the installation and upgrade burden outweighed the marginal control for a development and test platform.

Automatic imposes constraints, and one of them nearly blocked the project. Below Kubernetes 1.36, AKS Automatic refuses the creation of any `MutatingWebhookConfiguration` at all. Three of the platform's own dependencies — cert-manager, CloudNativePG, and ECK — require mutating webhooks to function. The considered alternative was to run an older Kubernetes version with those operators removed or replaced, which would have meant giving up certificate automation, the PostgreSQL operator, and the Elasticsearch operator simultaneously. The decision instead was to require 1.36 as a floor. Above that version the restriction is expressed as a `ValidatingAdmissionPolicy` scoped to `nodes`, `persistentvolumes`, `certificatesigningrequests`, and `tokenreviews`, which the platform's webhooks do not touch.

Automatic also leaves one seam unautomated: enabling Istio CNI chaining still requires an imperative `az aks mesh enable-istio-cni` call after the cluster is deployed. It is the sort of detail that a declarative system cannot quite absorb, and the CLI simply performs it.

Compliance baked into a chart, not patched on

Deployment Safeguards enforce pod security requirements at admission: workloads must run as non-root, must not allow privilege escalation, must drop all capabilities, must set a seccomp profile, and must declare CPU requests — the floor is `100m`.

Upstream OSDU Helm charts were not written with those constraints in mind. The obvious fix is to patch them at deploy time, with a Kustomize post-renderer or a HelmRelease `postRenderers` block. That was considered and rejected for a specific reason: a chart version bump can silently drop a patch, and the failure appears later as an admission rejection in an unrelated place.

Instead the platform maintains one local chart, `software/charts/osdu-spi-service/`, that bakes compliance into its templates: `runAsNonRoot`, `runAsUser: 1000`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, and `seccompProfile.type: RuntimeDefault`. Every OSDU

service is a `HelmRelease` against that one chart, overriding only image, environment, and resources. The chart also carries two init containers every service needs: one that builds a CA truststore, and one that fetches schemas.

The trade-off is honest: the platform now owns a chart it has to maintain. What it gets is that compliance drift shows up as a change to its own chart, in review, rather than as a surprise from upstream.

Node pools as application code

Karpenter decides what nodes to create. On AKS Automatic it is managed, but what it is told to create is the platform's business.

Those instructions — `karpenter.sh/v1 NodePool` objects and `karpenter.azure.com/v1beta1 AKSNodeClass` objects — are authored as Flux-managed workload manifests in `software/components/nodepools/`, reconciled by the `spi-nodepools` Kustomization very early, at what the layering calls Layer 0b.

Putting capacity definitions in Git alongside application manifests rather than in Bicep alongside the cluster was deliberate. Declaring them in Bicep would gate every node pool change on an infrastructure deployment and lose custom-resource-level review. Bootstrapping them imperatively from the CLI would reintroduce exactly the invisibility problem that GitOps exists to remove.

There are two pools rather than one, separated by taints — `workload=platform:NoSchedule` and `workload=osdu:NoSchedule` — so that middleware and OSDU services cannot displace each other. Both use the `AzureLinux` image family.

The project lists what it has not validated: the `agentpool` label is restricted at admission and cannot be set freely; a non-zonal node would create disks that cannot later attach to zonal VMs; and the five-minute consolidation delay is untuned for anything resembling production.

Availability zones are a discovery problem

A small decision with an outsized failure mode. Availability zone support is not uniform: a VM size may be unavailable in a zone, or restricted for a particular subscription, and the deployment only discovers this when resource creation fails partway through.

The platform resolves usable zones *before* creating anything. `_resolve_system_pool_zones` in `src/spi/azure_infra.py` queries `az vm list-skus` for the configured `SYSTEM_POOL_VM_SIZE` and fails preflight if the zones are unsupported or restricted for that subscription.

Requiring a manual per-environment override was rejected as pushing a discoverable fact onto the operator, and maintaining a per-region table was rejected as guaranteed to go stale. Failing hard

when the catalogue read itself fails was also rejected: if the query cannot be answered, the CLI warns and falls back to the template default rather than blocking a deployment on a transient API problem.

Bicep, and Why the CLI Stays Thin

Roughly fifty Azure resources have to exist before Kubernetes matters — a virtual network with private subnets and a NAT gateway, the cluster, managed identities, Key Vault, a container registry, a Gremlin account, a shared storage account, and then per-partition Cosmos SQL, Service Bus, and Storage, plus the role assignments that tie them together. Across partitions that reaches 24 blob containers, 14 Service Bus topics, and 14 subscriptions.

They are declared in hand-written Bicep, in three entry points:

- **infra/aks.bicep** — the cluster, managed Istio configuration, the virtual network and subnets, the NAT gateway, and the control-plane identity and its network role.
- **infra/main.bicep** — the OSDU and deploy identities, Key Vault, the container registry, Gremlin, common storage, the per-partition resources, and RBAC; optionally ExternalDNS and Application Insights.
- **infra/flux.bicep** — the AKS Flux extension and its Git configuration, which creates the two top-level Kustomizations.

Underneath are focused modules: `vnet.bicep`, `identity.bicep`, `keyvault.bicep`, `acr.bicep`, `cosmos-gremlin.bicep`, `storage-common.bicep`, `partition.bicep`, `rbac.bicep`, and a pair for the optional ExternalDNS identity and role.

Three alternatives were considered. **Terraform** was rejected because it requires somewhere to keep state and something to lock it, and the project did not want to operate that. **Azure Verified Modules** were rejected after a concrete failure: module version lag hid provider schema fields the platform needed, `proxyRedirectionMechanism` among them. A **pure az CLI orchestrator** was rejected because ordering has to be hand-maintained, there is no preview, and the prototype grew past a thousand lines.

That last point explains the division of labor. The CLI owns only three things Bicep cannot do well: creating the resource group, assigning the deployer a cluster-admin role, and writing a small number of runtime Key Vault secrets that are not known at template-authoring time. Everything else is declared. `spi up --dry-run` is `az deployment group what-if`, which means the operator can see the diff before it happens.

Two admissions come with this choice. The resource provider rejects `proxyRedirectionMechanism` at cluster creation, so it is set afterward. And the CLI must poll for role-assignment propagation before its first `kubectl` call, because Azure role assignments are eventually consistent and a cluster-admin grant is not immediately usable.

The largest one is stated flatly in the design guide: *An ARM deployment is not a transaction*. A failed deployment does not roll back. Some resources exist, some do not, and the recovery path is to fix the template and run it again — which works only because every template is written to be idempotent.

A digression on running anywhere

One decision in the register has nothing to do with Azure and is included here because it illustrates the project's temperament.

The CLI shells out to other tools constantly. On Windows, many of those tools — `az` among them — are batch shims, and a batch shim re-parses its command line through `cmd.exe`. Following CVE-2024-24576, passing arguments to a batch file safely became genuinely difficult: quoting that is correct for `CreateProcess` is not correct for `cmd.exe`, and a caret or a percent sign in an argument can change what command runs.

The platform routes every subprocess through a single function, `spi.shell.run_process`, which constructs an explicitly escaped `cmd.exe` command line using MSVCRT quote doubling plus a percent-neutralization trick. Per-call-site quoting was rejected as unmaintainable, bypassing the shims by invoking underlying executables was rejected as fragile, and simply refusing arguments containing `%` was rejected as too limiting.

The limits are documented rather than hidden: newline and NUL characters remain unrepresentable, a shim that re-parses its arguments a second time defeats the escaping, and `cmd.exe` caps a command line at 8,191 characters.

Ordering Is the Design

What GitOps means here

Once Azure resources exist, something has to put workloads into the cluster and keep them there. That job belongs to Flux, installed as a native AKS extension rather than by hand.

The choice of the extension over a manual `flux install` plus a hand-written `GitRepository` was made so that Azure owns component upgrades. One `fluxConfigurations` resource, declared in `infra/flux.bicep`, creates a `GitRepository` named `osdu-spi-stack-system` with a ten-minute sync interval, and two top-level `Kustomizations`: `stack` and `ingress`.

Splitting those two was deliberate. A single monolithic `Kustomization` would mean that any ingress change reconciles the entire service graph. Keeping them separate lets the addressing layer move without disturbing the workloads behind it.

Each root then creates children. The `stack` root creates the layered workload tree; the `ingress` root creates whichever of three addressing trees the environment selected.

Profiles: three sizes of truth

Not every environment needs all of it. The platform offers three profiles, and the distinction matters more than it first appears:

Profile	What Flux applies
<code>bare</code>	Empty stack and ingress trees; infrastructure and CLI bootstrap inputs only
<code>minimal</code>	Operators, middleware, and CA bootstrap — no OSDU services or initialization Jobs
<code>core</code>	The middleware substrate plus OSDU services, partition and entitlements initialization, schema loading, and reference APIs

The important caveat is that a profile selects *Kubernetes workloads*, not a smaller Azure estate. `spi up` still provisions AKS and the full set of PaaS templates for every profile, `bare` included. Choosing `bare` saves reconciliation time, not Azure resources.

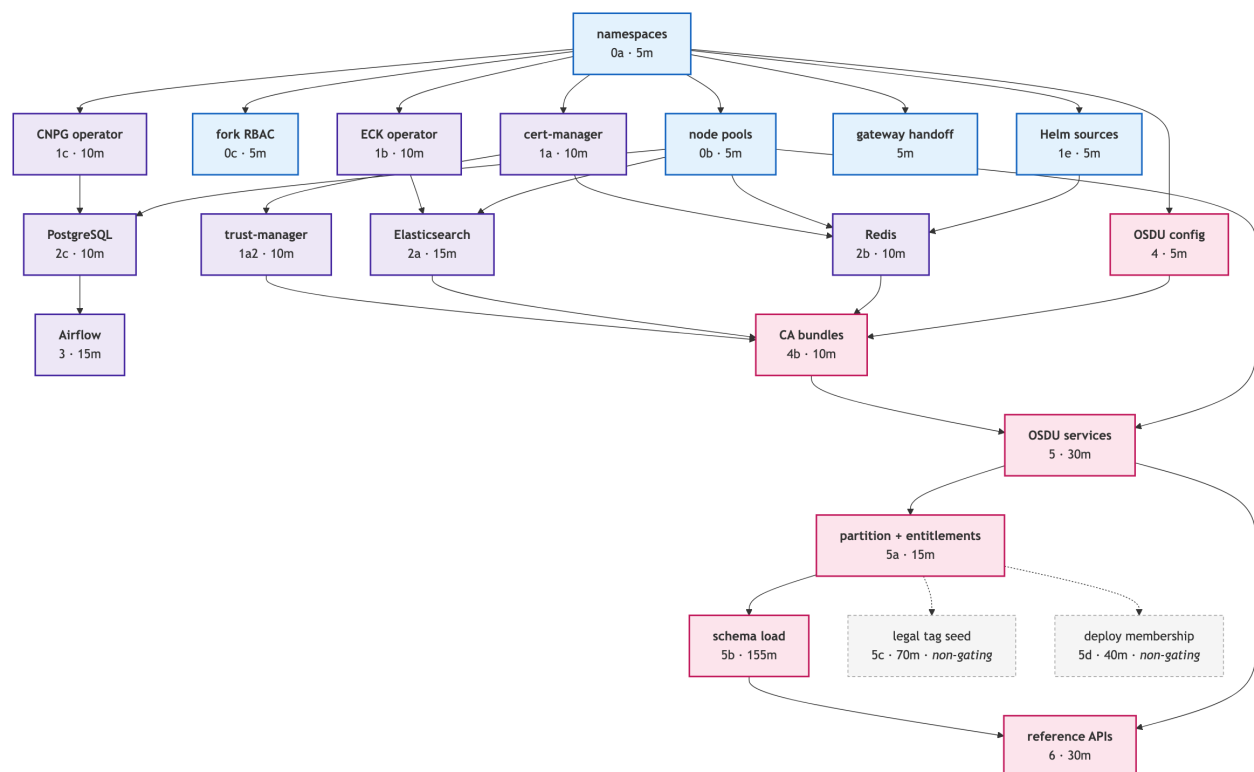
There was a fourth profile once, called `full`, and its removal is instructive. It was documented but never backed by a real manifest tree, so `--profile full` would deploy an entire Azure estate, spend the better part of an hour doing it, and then fail against a path that did not exist. The options were to build it or to delete it; the project deleted it, promoted the middleware-only tier to `minimal`, and made the profile list an enumerated `@allowed` value so an unsupported profile is rejected at parse time rather than forty-five minutes in.

Ordering is the design

Kubernetes does not have a general dependency system. A `CustomResourceDefinition` must exist before a custom resource can be created; an operator must be running before the resources it manages will do anything; Elasticsearch must be issuing certificates before anything can trust them.

The naive approach is one flat Kustomization containing everything, relying on apply order. The register rejects that in a single sentence: *kustomize apply order is not a dependency graph*. What it produces is CRD-not-found errors and CrashLoopBackOff cascades that look like bugs and are actually races.

Instead the core profile is wired as an explicit dependency graph, its layers labelled `0a` through `6`, each child Kustomization declaring `dependsOn` and `wait: true` so that its successors do not begin until its objects are genuinely ready. Kustomizations that do not depend on each other reconcile in parallel.



The core profile's dependency graph. Each box is a Flux Kustomization, labelled with its layer and its configured timeout. An arrow means the target waits for the source to report ready; boxes at the same depth reconcile in parallel. The two dashed branches are deliberately non-gating.

Reading it: namespaces exist first, and almost everything else hangs off them. Node pools, fork RBAC, the gateway ownership handoff, the shared Helm repository definitions, the three operators — cert-manager, ECK, CloudNativePG — and the OSDU configuration placeholder all fan out in parallel from that single root. trust-manager follows cert-manager because it needs the CRDs. Then

each middleware system waits on precisely what it needs and nothing more: Elasticsearch on the ECK operator and node pools, Redis on cert-manager, node pools, and the Helm sources, PostgreSQL on the CNPG operator and node pools. Airflow waits on PostgreSQL alone, and nothing waits on Airflow.

The graph then narrows. The CA-bundle bootstrap is the first real convergence point, waiting on trust-manager, Elasticsearch, Redis, and the OSDU configuration together — because it needs the CRD registered and both source CA secrets to exist. The OSDU services wait on that bootstrap and on node pools. Partition and entitlements initialization waits on the services. Schema loading waits on initialization, and specifically on the entitlements root groups it seeds: without them, the loader's call to `/schemas/system` returns 401.

The timeouts are not uniform, and their spread is a good map of where the time goes: five minutes for namespaces and node pools, ten to fifteen for operators and middleware, thirty for the OSDU service layer, forty for deploy-identity membership, seventy for the legal-tag seed, and one hundred fifty-five minutes for schema loading, whose Job carries an `activeDeadlineSeconds` of 9000 with reconcile headroom on top.

Two branches are deliberately non-gating. Legal-tag seeding and deploy-identity membership each hang off initialization as their own releases, carrying a `spi-stack.gating: "false"` label that keeps them out of the `spi status` readiness verdict. Nothing depends on either. The reason is stated in the manifest itself: a legal failure must not be able to block schema loading, because a missing legal tag is a data problem rather than a platform problem and should not hold an otherwise healthy environment hostage.

The reference APIs close the graph, waiting on both the OSDU services and the schema loader — the latter through a health check on the loader Job's `Complete` condition, which is the mechanism that lets one Kustomization depend on a *loaded* cluster rather than merely a running one.

One owner per object

A rule that sounds like bookkeeping and is actually a scar: every rendered Kubernetes object has exactly one Flux Kustomization inventory owner.

It was learned from the Gateway. Two Kustomizations rendered the same Gateway object, each recorded it in its own inventory, and each pruned the other's copy — producing an environment where ingress oscillated for no visible reason.

The fix required care, because Flux's default `MirrorPrune` behavior means that deleting a Kustomization deletes the objects in its inventory, including ones another Kustomization now legitimately claims. Rather than remove and rename in a single rollout, the old `spi-gateway` Kustomization was emptied and left in place as a handoff, with `prune: false` and `deletionPolicy: Orphan`, so that it releases its claim without destroying the object. The same

principle assigns the shared `bitnami HelmRepository` — used by both Redis and ExternalDNS — to exactly one owner, `spi-helm-sources`.

The deliberate pause

Here the platform departs from GitOps orthodoxy, on purpose.

Continuous reconciliation is normally the point: commit to Git, and the cluster converges. The platform does the opposite. `spi up` finishes by suspending the Flux `GitRepository` source — setting `spec.suspend: true` on `osdu-spi-stack-system` in `osdu-flux` — and then returns.

The reasoning is about what a development environment is for. An engineer debugging a failing service does not want the cluster shifting underneath the investigation because somebody merged to `main`. Updates should arrive when the operator asks for them.

The precise semantics are worth stating, because they are easy to get wrong. Suspension stops *fetching new commits*. It does not stop Flux: the controllers keep reconciling the cached revision, so drift within the cluster is still corrected. `spi reconcile --resume` clears the flag, `spi reconcile --suspend` sets it, and — a genuine sharp edge — plain `spi reconcile` does **not** temporarily resume a suspended source. An operator who edits Git, runs `spi reconcile`, and waits for a change that never arrives has met this behavior.

An `spi up` flag to skip suspension was considered and rejected as redundant, since `spi up` followed by `spi reconcile --resume` already expresses it. The suspension wait is capped at 120 seconds and a timeout is non-fatal; suspending does not interrupt a reconciliation already in flight.

The cost is acknowledged: hotfixes become a `kubectl apply` followed by a pull request to make the change durable, and Flux component versions are not pinned.

Identity, Which Is Two Different Problems

Authentication in this platform means two unrelated things that happen to share a word. Outbound: an OSDU pod needs to prove to Cosmos DB that it may read a record. Inbound: a user's HTTP request needs to arrive at a service with a trustworthy identity attached. They use different mechanisms and fail in different ways.

Outbound: credentials that do not exist

Because local auth is disabled everywhere, there is no connection string to store. What replaces it is Azure Workload Identity, and the pleasant consequence is that the cluster holds no Azure credential at all.

One user-assigned managed identity — `spi-stack-<env>-osdu-identity` — is federated against a Kubernetes ServiceAccount named `workload-identity-sa`, which the CLI creates in the namespaces that need it, annotated with the identity's client ID and tenant ID. A pod opts in by using that ServiceAccount and carrying the label `azure.workload.identity/use: "true"`.

From there the exchange runs:

1. The AKS Workload Identity webhook sees the label and mutates the pod, projecting a ServiceAccount token into it as a file and injecting `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, and `AZURE_FEDERATED_TOKEN_FILE`.
2. The Azure SDK inside the application reads that file and presents the token to Entra ID as proof of identity.
3. Entra ID, which has been told to trust tokens from this cluster's issuer for this subject, returns an Azure access token.
4. The Azure resource authorizes the request against the identity's role assignments.

The projected token's audience is `api://AzureADTokenExchange`. It is a short-lived file, rotated by Kubernetes, never stored, and never visible in a Secret.

One identity is shared by all OSDU services rather than one per service. Per-service identities were considered and rejected because the role surface does not differentiate enough to justify the federation and RBAC volume. The cost is stated rather than hidden: every OSDU service shares one access envelope, so there is no Azure-layer blast-radius containment between them.

Inbound: rewriting identity at the door

The Azure provider code in OSDU does not read JWTs. It reads headers — specifically `x-app-id` and `x-user-id` — because that is the contract `core-lib-azure` and the Spring filter chain were built around.

That creates an obvious danger: if a service trusts a header, anyone who can set that header is anyone they like. The platform's answer is to make it impossible for a caller to set those headers at all, by having the mesh strip and rewrite them at the edge of every pod.

Three objects in `osdu` do the work:

- **RequestAuthentication/spi-osdu-jwt-authn** validates a presented JWT against the configured Entra issuers and audiences. If a token is present and invalid, the request is rejected. If no token is present, the request continues unauthenticated — validation is not authorization.
- **EnvoyFilter/spi-osdu-identity-filter** runs a Lua filter on the inbound sidecar path. It first *removes* any incoming `x-app-id` and `x-user-id` headers, then repopulates them from the validated token's claims.
- **PeerAuthentication/spi-osdu-mtls** sets mesh peer authentication to `PERMISSIVE`, which is what allows bootstrap Jobs without sidecars to talk to services that have them.

The claim mapping carries a detail that explains an otherwise baffling piece of the CLI. `x-app-id` comes from the token's own application identifier, which is the `appid` claim on a v1 Entra token and `azp` on a v2 token. `x-user-id` follows issuer-specific claims — `oid`, `unique_name`, or `upn`.

Because of that, when `spi token` mints a token for hand-testing, it deliberately posts its client assertion to the `v1` endpoint, `https://login.microsoftonline.com/<tenant>/oauth2/token`. A v2 token would carry `azp` but omit `appid`, and the projection would come out differently. The assertion itself is a ServiceAccount token obtained with `kubectl create token` for audience `api://AzureADTokenExchange`, with a ten-minute lifetime.

Four alternatives were rejected, and the reasons are a short lesson in authorization design. Substituting the OSDU identity's client ID for any caller with a management audience would have solved the bootstrap case and simultaneously created a general impersonation path. Pairing `RequestAuthentication` with an `AuthorizationPolicy` keyed on JWT claims would be the idiomatic Istio answer, but the Azure provider chain reads headers, not the request principal, so it would not actually protect the code path in use. Per-service default-deny policies were judged too broad a blast radius for a development platform. And pre-populating entitlements to sidestep the whole question was rejected because it bypasses the auth chain and couples bootstrap to entitlements internals.

The failure mode is worth recognizing on sight: if the JWKS endpoint is unreachable, or the `RequestAuthentication` has drifted, or a sidecar is running a different revision than expected, the projection produces an empty value and downstream logs show `app-id=` with nothing after it.

Three planes of secrets

Secrets are split by class, and the split is the decision:

Class	Where it lives	Why
Azure PaaS access	Nowhere — Entra ID tokens, minted per request	A stored credential is a credential that can leak
Stored configuration values	Azure Key Vault, written at deploy time by Bicep	Deploy-time guarantees; read by the SDK under Workload Identity
In-cluster middleware passwords	Kubernetes Secrets	Elasticsearch and PostgreSQL need a password; there is no Entra path to them

Kubernetes Secrets for PaaS credentials were rejected because they are readable by anything with cluster read access. Mounting every secret through the Key Vault CSI driver was rejected because SDK reads under Workload Identity keep the access path in code, matching how the upstream Azure provider modules already work. Writing every Key Vault secret after deployment from Python was rejected because it gives up Bicep’s deploy-time guarantees for values that are perfectly well known at template time.

The honest gap here is rotation. The documentation states that there is no supported end-to-end password-rotation command in the CLI, and that rotation is not automated.

Moving a certificate authority across a namespace

A small problem with a tidy solution. Elasticsearch and Redis run in `platform` and present TLS certificates signed by their own self-signed certificate authorities. The OSDU services that connect to them run in `osdu`. A Kubernetes Secret does not cross namespaces.

The platform uses trust-manager `Bundle` resources to mirror exactly two CAs — `redis-ca-cert` and `elastic-ca-cert`, sourced from `redis-tls-secret` and `elasticsearch-es-http-certs-public` — from `platform` into `osdu`. Two bundles, two authorized target Secret names, and rotation re-syncs automatically.

Three alternatives were rejected: a Flux-managed Job with its own managed identity, which needs identity and RBAC plumbing and only runs when triggered; general-purpose replicators such as `kubernetes-replicator` or Reflector, which require broad cluster-scoped Secret permissions; and an imperative copy from the CLI on every `spi up`, which is brittle and sits outside GitOps entirely.

One consequence needs a manual override. Istio would normally originate its own mTLS to Redis, which conflicts with Redis’s own TLS. A static `DestinationRule` named `redis-disable-mtls` sets `trafficPolicy.tls.mode: DISABLE` for `platform-redis-master.platform.svc.cluster.local`.

Getting In

Three products, not one with switches

Exposing the platform is not one problem. Somebody evaluating it on a laptop, a team sharing an environment with a real domain, and an engineer debugging inside a private network want genuinely different things.

The platform ships three ingress modes, selected with `--ingress-mode` or `SPI_INGRESS_MODE`, defaulting to `azure`, each a complete manifest tree under `software/stacks/osdu/ingress/<mode>/`:

Mode	Address	Edge TLS	For
azure	<code><label>.<region>.cloudapp.azure.com</code>	Let's Encrypt, one hostname	Development and testing without owning a DNS zone
dns	Environment-prefixed names in an Azure DNS zone you own	Let's Encrypt, separate hostnames per subdomain	Team environments
ip	The gateway's public IP	None	Isolated debugging only

In `dns` mode, ExternalDNS watches HTTPRoutes and maintains three subdomains — `osdu`, `kibana`, and `airflow`.

The alternative was a single mode with conditional overlays. It was rejected because a matrix of conditionals is hard to review and makes `what-if` diffs opaque — a recurring theme in this project, which consistently prefers three legible things to one clever thing. Always provisioning an Azure DNS zone was rejected because many users do not own one, and a Terraform-first DNS path was rejected because ExternalDNS's continuous reconciliation tracks HTTPRoute changes better.

Binding to somebody else's gateway

AKS Automatic's managed Istio add-on already runs an ingress gateway and owns the `aks-istio-ingress` namespace, where cluster policy blocks the platform from writing freely.

So the platform does not create an ingress workload. It creates a `Gateway` named `spi-gateway` that *binds* to the add-on's existing `aks-istio-ingressgateway-external` Service by hostname address. There is deliberately no `per-gateway` Service of its own.

In `azure` mode a DNS label has to be set on that Service, which the platform may not simply edit. The workaround is precise: `spi-ingress-dns-label` is a partial Service manifest owning only the `azure-dns-label-name` annotation, applied by Flux with `prune: false` so that removing it never

deletes the Service it annotates. Setting the DNS label on the public IP instead was rejected because the node resource group carries a deny assignment that blocks it.

A certificate in the wrong namespace

cert-manager needs to write a `Certificate`, a `CertificateRequest`, and eventually a TLS `Secret`. The Gateway that consumes that Secret lives in `aks-istio-ingress` — where those writes are denied by policy.

Exempting cert-manager from policy was rejected, and so was relocating cert-manager into a managed namespace. Instead the `Certificate` and its resulting Secret live in `platform`, and a `ReferenceGrant` authorizes the Gateway in `aks-istio-ingress` to read across the namespace boundary. The HTTP-01 solver routes live in `platform` too.

This chapter's most useful warning came out of that work: a `Certificate` whose status is never updated can still pass a Flux health check, producing a Kustomization that reports ready while HTTPS never opens. Readiness of the object is not readiness of the service.

Six places an endpoint can fail

Because a request crosses DNS, a load balancer, TLS termination, mesh routing, and an application, “it does not work” is not a diagnosis. The ingress guide enumerates the six failure points in order, and the order is the debugging procedure:

1. DNS lookup fails.
2. The connection fails.
3. The TLS handshake fails.
4. The gateway route does not match.
5. The backend is unavailable or returns a 5xx.
6. The application returns 404.

The last one carries the insight. A completed HTTPS request that returns 404 has already proven DNS resolution, TCP connectivity, TLS negotiation, and gateway routing. It is the *best* failure on this list, and reading it as a total outage sends debugging in exactly the wrong direction.

Two rough edges are documented rather than papered over. In `ip` mode there is no HTTPS and no middleware UI access at all. And there is a genuine input-handling bug, stated verbatim in the guide: *explicitly passing `--dns-zone` sets the name but skips discovery of the zone's resource group. The CLI has no corresponding resource-group option, and Bicep needs both values.* The instruction that follows is unusually direct — use automatic discovery when the subscription has one zone, and do not treat explicit selection among multiple zones as a complete deployment path until that input handling is fixed.

Making an Empty OSDU Useful

A deployed OSDU answers no useful question. The services are running, the databases are empty, and every request fails authorization because nothing has been told who anybody is. Three things must exist first: partitions, entitlements, and schemas.

Bootstrap as data

All of it is done by one Flux-managed Helm chart, `software/charts/osdu-spi-init/`, which renders four one-shot Jobs per partition:

1. `partition-init` creates the partition record that every other service looks up.
2. `entitlements-init` creates the root groups.
3. `legal-init` seeds a default legal tag.
4. `entitlements-members` adds the deploy identity to `users`, `users.datalake.ops`, `users.datalake.admins`, `users.data.root`, and `users.datalake.delegation` for every partition — creating the delegation group and `users.datalake.impersonation` first if the tenant bootstrap has not.

Three alternatives were rejected. An imperative CLI step reopens the hidden-dependency and invisibility problems. Stamping a separate Flux Kustomization per partition duplicates wiring. Partition-prefixed chart values were rejected for a concrete reason: they double-prefix the stored values and break lookups.

The Jobs are written to be re-runnable, which means treating HTTP status codes as a state machine rather than as success or failure: `201` and `204` are success, `409` means somebody already did it and is therefore also success, and `403` is retried. That last one is a genuine oddity worth knowing — a partition lookup returns `403` both before the partition exists and after it exists but before entitlements has caught up, so the same code means two different things at two different times, and only retrying distinguishes them. Entitlements cache lag makes retries necessary generally.

One small deliberate choice: `ttlSecondsAfterFinished` is omitted, so completed Jobs and their logs stay in the cluster for inspection instead of vanishing on success.

Loading 1,386 schemas

The schema service needs the community's shared schemas — roughly 1,386 of them — and until they are loaded, most records cannot be validated or created.

Loading runs as a Flux-managed Job in `osdu` at Layer 5b, reusing `workload-identity-sa` so that it authenticates the same way every service does. It is the longest single operation in the platform: the

Job carries an `activeDeadlineSeconds` of 9000, and the Kustomization that owns it allows 155 minutes. As with bootstrap, `already exists` responses are not fatal.

Three alternatives were rejected, and the first has the best line in the register. A long-running Deployment that loads schemas and then sleeps forever was rejected because *it lies to Kubernetes* — a Deployment claims to be a service that should be kept running, and a loader is not. A Terraform `null_resource` with `local-exec` was rejected because it reintroduces the CLI-tail problem, where the operator's laptop has to stay connected for the platform to finish converging. A home-grown Python loader reading schemas from Git was rejected because it duplicates work the community's own CI already does.

The residual risk is supply-chain rather than logic: the loader image's tag depends on the OSDU registry's retention policy, and mirroring it into the platform's own container registry is noted as a follow-up if that retention ever becomes a problem.

The image lock

Which image does each service run? The answer is a single ConfigMap, and it is worth understanding carefully, because Book Three is largely a story about writing to it.

`osdu-image-lock` lives in `osdu-flux` and holds resolved image references for every service — repository, tag, digest, and a fully-qualified reference, under keys such as `PARTITION_IMAGE`, `PARTITION_IMAGE_TAG`, `PARTITION_IMAGE_DIGEST`, and `SCHEMA_LOAD_IMAGE_REF`. Flux substitutes those values into the rendered manifests via `postBuild.substituteFrom`, so no `HelmRelease` contains an image tag at all. `src/spi/images.py` resolves the entries; `spi reconcile --refresh-images` is the only operation that moves canonical ones.

Six alternatives were considered, and the list reads as a catalogue of everything that goes wrong with image management:

- **Pin tags in each service's `HelmRelease.values`** — every refresh becomes N file edits.
- **Follow `latest` with `reconcileStrategy: Revision`** — surprise upgrades, in a platform whose whole point is reproducibility.
- **Commit a static `osdu-image-lock.yaml`** — refreshes still require N Git edits.
- **A Helm post-renderer or Kustomize patch chain** — pin logic migrates into render-time templates, where it is hard to see.
- **Pin by suspending the owning Kustomization and patching the `HelmRelease`** — freezes every sibling service along with the one being tested.
- **Pin by pointing services at an image branch** — validation needs exactly one service moved and thirteen held, which this cannot express.

That final requirement is the one that matters, and it is precisely the requirement Book Three arrives with: move one service's image, leave every other service exactly as it was, and be able to put it back.

A ConfigMap of substitution variables is the only one of the seven options that does it without touching Git or disturbing a neighbor.

Two operational notes travel with it. A tag can be pruned from a registry, so a tag-based pin may break on a later reconcile where a digest would not. And completed Jobs cannot be updated in place, so moving an image that a Job uses requires `force: true`.

Fifty Minutes, Start to Finish

Everything so far has been structural. This chapter is the same system as a timeline, because the durations are the part that surprises people.

A full deployment takes roughly 45 to 50 minutes. It is dominated by things the platform does not control: about 30 minutes for AKS Automatic to provision, and another 10 to 15 for the Flux extension to install.

The order of events

Before anything is created. `spi check` verifies prerequisites; zone resolution queries the subscription's SKU catalogue for usable availability zones; `spi up --dry-run` can render an `az deployment group what-if diff`.

Azure. The resource group is created by the CLI. Then the Bicep deployments: the cluster, the network, identities, Key Vault, the registry, Gremlin, common storage, and per-partition Cosmos, Service Bus, and Storage. The cluster is the long pole.

The cluster. The CLI assigns itself a cluster-admin role and then polls for that assignment to propagate — Azure role assignments are eventually consistent, and the first `kubectl` call will fail if it does not wait. It enables Istio CNI chaining, creates `osdu-flux`, writes the bootstrap ConfigMaps and the credential seed, and creates `workload-identity-sa` in the namespaces that need it.

Flux activation. The extension is installed and the `fluxConfigurations` resource creates `stack` and `ingress`.

The pause. The CLI waits up to 120 seconds for the Git source to report ready, suspends it, and returns.

Meanwhile, Flux is still working. This is the part that catches people. `spi up` has exited successfully and the platform is nowhere near ready. Operators are starting, Elasticsearch is forming a cluster, certificates are being issued, OSDU services are rolling out, bootstrap Jobs are retrying through entitlements cache lag, and the schema loader may have well over an hour of work in front of it.

Five milestones, not one

Because of that, “is it up?” has no single answer, and the platform deliberately refuses to invent one. Readiness is reported as distinct facts: whether the CLI finished, whether every Kustomization is reconciled, whether bootstrap completed, whether the APIs answer, and whether the environment may currently be deployed to.

The last of those is a separate question from all the others, and the next book depends entirely on its being separate.

The measured and configured durations are worth separating carefully, since conflating them is the most common way to get this wrong:

Value	Kind
45–50 minutes total provisioning; ~30 min AKS; 10–15 min Flux extension	Measured
120 seconds waiting for the Git source to report ready	Configured, non-fatal
150 minutes schema-load Job deadline; 155 minutes for its Kustomization	Configured
230 minutes for cold-cluster Flux readiness in CI	Configured budget
45 minutes for an ordinary <code>spi down</code> to complete deletion	Configured deadline
40 minutes per <code>spi reconcile --refresh-images</code> wait	Configured

The design guides repeat one warning about this table, and it applies to every number in this book: these phases overlap, and summing them produces a wall-clock estimate that is simply wrong.

Coming back down

Teardown is where an unobvious decision lives.

`spi down` deletes the resources in the environment’s group individually, and it is slower and less atomic than deleting the group would be. It deliberately leaves three things behind: the user-assigned managed identities, the resource group itself, and the group’s tags.

The reason is repository onboarding, which Book Three explains in full. External repositories are granted trust by federating a credential against a managed identity, and they hold that identity’s client ID in their own configuration. If a reset produced a new identity, every onboarded repository would need its client ID re-stamped. The alternative considered was keeping identities in a separate permanent resource group, which was rejected as more moving parts for the same outcome.

So `spi down` is scoped to leave the identity standing, and `spi up` reuses it. Five connection values are given to an onboarded repository; four of them are stable across a full rebuild.

`spi down --purge` is the real deletion: it first removes external role grants — such as a DNS Zone Contributor assignment in another resource group — confirms that removal, and only then deletes the whole group and the identities with it. Purge needs subscription-wide permission to enumerate assignments; if discovery fails, permissions are missing, or a grant cannot be confirmed removed, the purge stops and retains the identities rather than orphaning a grant.

The lifecycle verbs above environment level are `status`, `refresh`, `upgrade`, `reset`, and `teardown`. `Refresh` changes runtime state only. `Upgrade` re-runs `spi up` at a new tag. Only a full reset — `spi down` followed by `spi up` — clears both cluster and Azure data state, and it is run weekly rather than on demand.

Two honest notes close the chapter. A `Ready=True` result can be reported against stale cached artifacts after the Git reference changes, which is a direct consequence of the deliberate pause. And a failed lifecycle run leaves the environment's `maintenance` flag set — deliberately, so that a broken environment stops accepting deployments until a human looks at it. That flag is the first thing Book Three checks.

Book Three: The Handshake

Where the two systems meet

Book One produced a container image and could not tell you whether it worked. Book Two produced a running environment and could not tell you where a trustworthy image comes from. This book is about the forty minutes in which those two facts are resolved against each other.

The transaction is easy to state and surprisingly hard to build. A fork has a candidate image. It borrows a slot in a shared environment somebody else operates, puts its own image in that slot temporarily, runs its acceptance suites against the live APIs, and puts back what was there before — whether the tests passed, failed, or never ran.

Every word in that sentence is load-bearing. *Borrows* means the environment is not owned by the fork and must be returned intact. *Temporarily* means the change has to be undoable by a process that may itself crash. *Shared* means another service may be doing the same thing at the same time. *Whether the tests passed or not* means the restore path cannot be conditional on success.

What makes this the climax of the book rather than an appendix is that it is the only place where both systems' design decisions have to agree. The engineering system's refusal to store environment values, the platform's refusal to store credentials, the image lock's substitution model, the deliberate GitOps pause, the readiness contract, the managed identities that survive teardown — each was introduced separately, for its own reasons, and each turns out to be a precondition for this handshake. Read alone, several of them look like fussiness. Read here, they look like the same design.

Seven chapters: why the problem exists at all, how trust is established before anything runs, the contract a fork uses to describe its own tests, the borrow, the proof, the return, and an honest accounting of which parts are built.

Two Systems Meet

The question every test asks first

An OSDU acceptance test suite is not self-contained. Before it can assert anything, it must know where the service is, how to authenticate to it, and which identifiers to use: a gateway URL, a data partition, a tenant, a bearer token, sometimes a legal tag.

The community's own subgroup documentation names the root cause of a decade of accumulated pain in one sentence, which this project quotes:

| *"The tests are in the repository; the knowledge of how to run them is not."*

Upstream kept that knowledge in four cloud providers' pipelines. Severing those pipelines — which is exactly what Book One's filter does — orphans the tests. They exist, they compile, and nobody can run them.

So the engineering system inherits a problem it created. It must supply the missing knowledge without hard-coding it, for eight service forks, plus customer forks pointing at environments Microsoft has never seen.

Five constraints that rule out the obvious answer

The obvious answer is to store the environment details in each repository as variables. Each of the following makes that worse.

The environment is disposable. As Book Two established, the shared environment is torn down and rebuilt on a weekly cadence. Any environment value copied into a repository is stale by construction.

The machinery is multiplied. The same workflows synchronize from the template to eight service forks and onward to customer mirror forks pointing at *their own* environments with *their own* credentials. Nothing environment-specific or Microsoft-specific can be baked in.

Developers come first. The same tests must run against a personal Stack instance during development, with no CI in the path.

Releases must stay provable. "Run the tests that shipped with release X against environment Y" must remain a one-command operation months later.

CI must stay fast and honest. It must fail in seconds with a typed reason when the environment is not ready, and must never report green for a suite that did not run.

A working prototype had already demonstrated the failure mode. It pushed roughly sixteen repository variables into each fork, of which six went stale on every weekly rebuild, with no

reconciliation loop — and it carried a sticky `DEPLOY_VALIDATED` boolean that never expired, meaning it kept vouching for an environment that no longer existed.

The project distills the requirement to a heading: **no environment values in the repository**.

Three contracts, three owners

The answer separates what is known from who knows it.

Contract	Owner	Content
Facts — <code>spi info --json</code> , <code>spi status --json</code>	<code>osdu-spi-stack</code>	What the environment knows about itself: endpoints, tenant, data-plane client ID, partitions, Key Vault name, deployability, maintenance state, pinned services, seeded loads. Read per run, cached nowhere.
Descriptor — <code>.spi/service.yaml</code>	each service fork	What this service's suites need, symbolically. Reviewed in the same pull request as the test change.
Machinery — actions and <code>validate.yml</code>	<code>osdu-spi</code> (template)	The resolver joining descriptor against facts, the acceptance image build, the deploy lane, the required-check summaries. Synchronized to every fork.

The facts envelope is versioned — `apiVersion: spi.osdu.dev/v1` — so the contract can evolve without silent breakage.

The descriptor is fork-owned for a specific reason. The community acceptance test module is upstream-owned and regenerated on every synchronization, so nothing fork-specific can live inside it. The contract lives *beside* it, in `.spi/`, which the filter excludes from synchronization and CODEOWNERS protects. This is Book One's ownership-runs-through-the-tree rule doing work it was designed for.

What the environment refuses to tell you in one place

One design decision on the platform side is easy to miss and shapes everything downstream: `spi status` and `spi info` are separate commands, on purpose.

`spi status --json` answers operational questions — is this environment reconciled, is it deployable, is it in maintenance, what is not ready. `spi info --json` answers factual ones — what are the endpoints, the partitions, the tenant, the client IDs.

Folding the endpoints into the status envelope was considered and rejected, as was a third command called `spi facts`. The reasoning is about caching and freshness: status is volatile and must be re-read on every decision, while facts are stable for the lifetime of a deployment. Keeping them separate

means a consumer cannot accidentally treat a stale readiness verdict as current because it arrived alongside a still-valid endpoint.

Also rejected: letting consumers read `kubectl get kustomizations -o json` directly. That would have worked, and it would have frozen the platform's internal Flux layout into an external contract that eight repositories depend on.

The envelope is worth seeing in full, because Book Three's remaining chapters are a story about reading it:

```
{
  "apiVersion": "spi.osdu.dev/v1",
  "ready": true,
  "deployable": true,
  "reason": { "code": "...", "message": "...", "resource": "..." },
  "suspended": true,
  "maintenance": false,
  "kustomizations": { "total": 0, "ready": 0, "notReady": [] },
  "environment": {
    "name": "...", "stackVersion": "...", "resolvedCommit": "...",
    "profile": "...", "deployedAt": "...", "cliVersion": "..."
  },
  "images": { "branch": "...", "count": 0, "pinnedServices": [] },
  "baseUrl": "..."
}
```

Three fields repay attention. `ready` is deliberately **not** the same as `kustomizations.ready == kustomizations.total`. `deployable` is a separate verdict that can be false while `ready` is true — because the environment is in maintenance, because there is no deploy record, or because the entitlements bootstrap failed or has not finished. And `reason.code` is drawn from a closed set: `kustomization_not_ready`, `maintenance`, `missing_deploy_record`, `bootstrap_failed`, `bootstrap_pending`.

Folding `maintenance` into `ready` was explicitly rejected. An environment under maintenance may be perfectly healthy; it is simply not accepting deployments right now, and a consumer deserves to be told which of those two things is true. The command's exit code carries the same verdict for callers that do not parse JSON: `0` when deployable, `2` when not.

That single separation is what replaced the sticky `DEPLOY_VALIDATED` boolean. The old design asked a repository to remember a verdict. The new one asks the environment to state one, every time.

Trust Before Anything Runs

Nothing in the previous chapter can happen until a specific, durable relationship exists between a fork repository and an environment. Establishing it is a deliberate, human-initiated act, and the shape it takes explains several decisions from Book Two that were left unexplained there.

Onboarding is a grant, not a configuration

An operator runs one command against the environment:

```
spi onboard <service> --repo <org>/<fork>
```

That adds a federated credential to the environment's deploy identity — the user-assigned managed identity `spi-stack-<env>-deployer` that `spi up` created — trusting GitHub tokens whose subject is exactly:

```
repo:<org>/<fork>:environment:spi-stack
```

Read that subject carefully, because it is the security boundary of the entire handshake. GitHub signs it, Azure verifies it, and it names three things: which repository, and that the token was issued to a job running in a GitHub Environment named `spi-stack`. This is the reason the `spi-stack` environment exists at all in the fork's workflows — not to store configuration, but so that GitHub will assert its name inside a token.

A generic `pull_request` subject was considered and rejected as far too broad. The environment subject admits every branch of the trusted repository, which is deliberate: write access to the fork is the real boundary, and a pull request from an outside repository never receives an OIDC token in the first place.

The alternative architecture — one managed identity per fork, held in a separate permanent resource group — was rejected as more moving parts for the same result. That choice is precisely why Book Two's teardown leaves identities standing: with a single shared deploy identity, a rebuild that changed its client ID would invalidate every onboarded repository at once.

There is a hard ceiling worth knowing. Entra allows twenty federated credentials per managed identity, and the cluster itself consumes one, so **at most nineteen repositories** can be onboarded to a single environment.

Two smaller sharp edges: `spi onboard` refuses `minimal` and `bare` profiles, because there are no OSDU services in them to deploy to. And a repository that is deleted and recreated under the same name receives a new internal ID, so the federated credential no longer matches and it must be onboarded again — a confusing failure, since nothing visible changed.

What the credential is allowed to do

Trust alone is not authority. The deploy identity's Azure roles are deliberately minimal: **AKS Cluster User** and **Key Vault Secrets User**. Nothing else.

Its Kubernetes permissions are narrower still, expressed as two namespace-scoped Roles bound through the `spi-cluster-config` ConfigMap rather than written into manifests:

- **spi-fork-deployer** in `osdu-flux` — patch on the image lock, restricted by `resourceNames` to `osdu-image-lock` and nothing else.
- **spi-fork-verifier** in `osdu` — read-only on deployments, pods, and logs.

There is no create verb, no delete verb, and no access to secrets at all. A compromised fork workflow can change which image one service runs and read that service's logs. It cannot create a pod, delete a resource, read a Kubernetes Secret, or touch another namespace.

Writing those RoleBindings into the stack manifests with principal IDs supplied by pull request was rejected — it would make onboarding a code change reviewed by the wrong people. Binding an Entra group instead of a principal was rejected for a concrete technical reason: managed-identity tokens carry no group claims, so a group binding would simply never match. An escape hatch of the form `SPI_SKIP_GUARD=1` was considered and rejected outright. And supplying acceptance credentials as Kubernetes Secrets was rejected, which is why tokens are minted per run instead.

Trusted and canonical are different questions

Here is the distinction that most repays understanding, because the two concepts are easy to conflate and the platform keeps them rigorously apart.

Trust says: this repository is allowed to borrow this environment. It is the federated credential.

Canonical source says: when nobody is borrowing, where does this service's image normally come from? The answer is either `community` — the upstream-published image — or a specific fork's GHCR package at `ghcr.io/<lowercase-owner>/<service>`.

Onboarding a repository grants trust and *nothing else*. It does not change which image the environment runs by default. Promoting a fork to be the canonical source is a separate, explicit act:

```
spi onboard --canonical-source fork
spi onboard --canonical-source community
```

Reverting to `community` does not revoke trust; the repository can still borrow the environment to test, it simply no longer supplies the default image.

The record of that choice is durable and lives outside the cluster, as a resource group tag named `spi-source-<service>`. A missing tag means community. That placement is deliberate: a cluster can be

destroyed and rebuilt, and the source policy has to survive it — the same reasoning that keeps the managed identities alive through `spi down`.

Several tempting shortcuts were rejected. Deriving the source from the credential roster — treating “onboarded” as “canonical” — is exactly the conflation this decision exists to prevent. One fleet-wide flip was rejected because services migrate one at a time. Dual-source fallback, where a missing fork image quietly falls back to community, was rejected as the worst option of all: it would mean an environment silently running different code than the operator believes. A missing or private package is therefore a hard resolution failure, not a fallback.

One risk is recorded and not yet mitigated: GHCR retention is thirty days, and a refresh cycle that stalls long enough can let a retained canonical version age into contents that no longer exist.

The environment on the other end

A final piece of context, because it explains why the borrowed environment is stable enough to borrow.

The shared environment used by fork CI deploys a **release tag, never a branch**. Its entire declaration lives in one reviewed file, `ops/environments/shared.yaml`, which names the stack version and lists the fork repositories with their canonical source setting:

```
forks:
  - service: partition
    repo: <org>/<fork>
    canonicalSource: community
```

Tracking `main` continuously was rejected as putting CI for eight repositories at the mercy of every merge. Pinning to a commit SHA was rejected as unreadable. Moving the declaration to a separate environment repository was rejected because it separates the change from its review.

This pin is also why the fork’s workflow does something that looks redundant and is not — it matches its client to the environment’s version, which is the first real step of the borrow.

The Descriptor

A fork describes its own tests in `.spi/service.yaml`. The descriptor declares named suites, each binding environment variables to *symbols* rather than values:

```
schemaVersion: 3
service: { name: partition, archetype: java-maven-azure }
tests:
  acceptance:
    type: maven
    path: partition-acceptance-test      # upstream module, kept by the filter
    mavenArguments: [test]
    bindings:
      HOST: { source: gateway }
      DATA_PARTITION_ID: { source: partition }
      PRIVILEGED_USER_TOKEN: { source: token }
    timeoutMinutes: 15
```

The source vocabulary is **closed**: `gateway`, `partition`, `openid`, `tenant`, `legalTag`, `domain`, `keyvault:<name>`, `static`, `template`, `user`, `token`, `memberToken`, and `noAccessToken`. An unknown source kind is a hard failure naming the key — the same halt-on-the-unknown rule the filter follows in Book One, applied by a different component for the same reason.

Certain environment names are refused outright, because a descriptor that could set them could hijack the runner: exact names like `AZURE_CLIENT_ID`, and anything prefixed `ACTIONS_`, `GITHUB_`, `RESOLVER_`, `RUNNER_`, or `SPI_STACK_`.

Resolution precedence is explicit environment, then facts, then a declared default, with `template` rendering last. That ordering is what lets an identical descriptor run in CI, against a developer’s personal environment, or pointed at a service running on a laptop.

The resolver’s exit codes are typed, and the distinction matters more than it appears: **2** is a descriptor violation, **3** is environment-not-ready, and **4** is an infrastructure contradiction. A test failure is none of these. Conflating “the environment was down” with “the code is broken” is precisely the ambiguity the taxonomy exists to remove.

What the repository is allowed to know

The entire per-repository configuration surface is five values.

Value	Kind	Why it may live in the repository
AZURE_CLIENT_ID , AZURE_TENANT_ID , AZURE_SUBSCRIPTION_ID	secrets	The organization’s OIDC identity for its own environment. Stable across rebuilds.

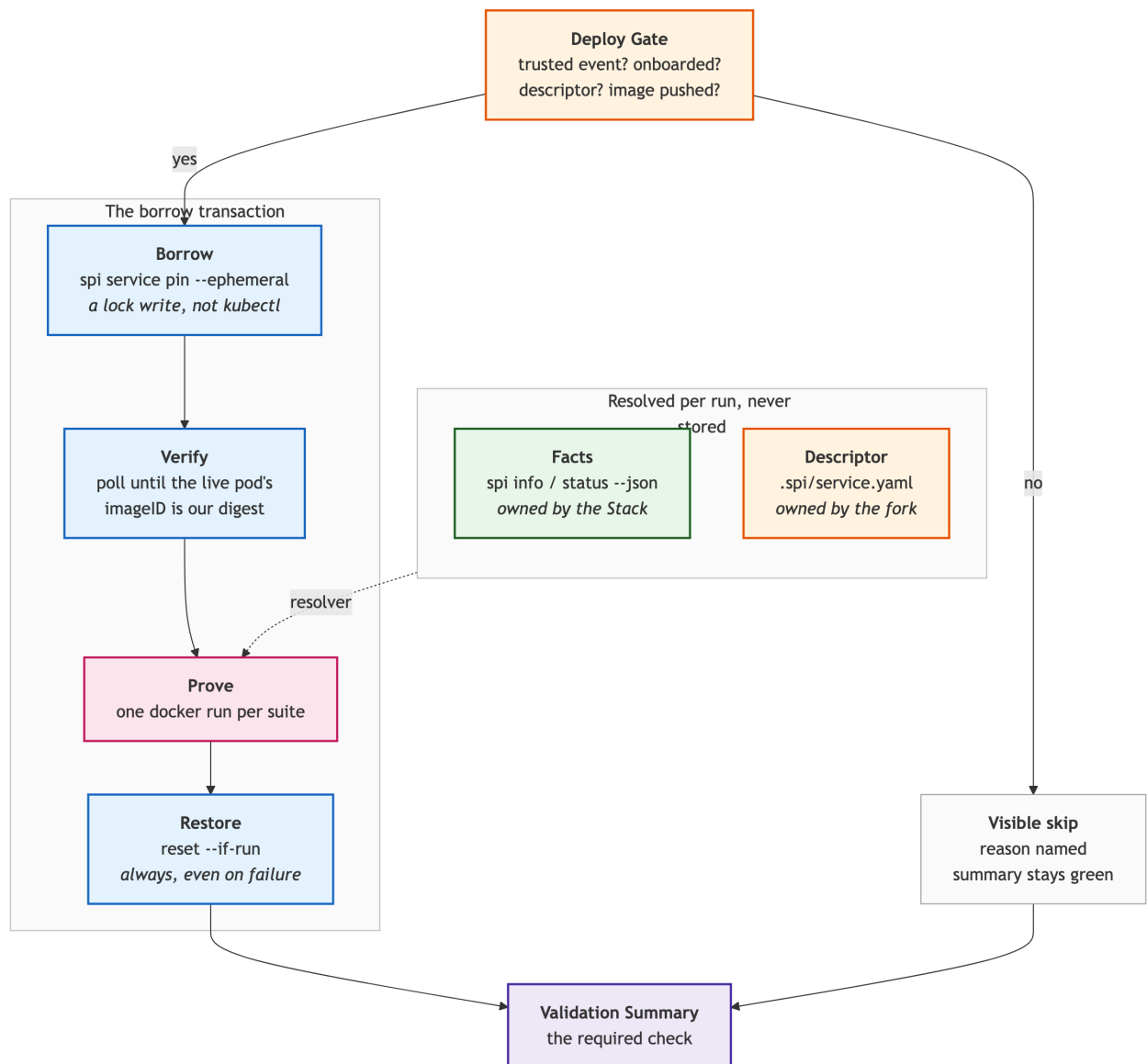
Value	Kind	Why it may live in the repository
SPI_STACK_RESOURCE_GROUP , SPI_STACK_CLUSTER	variables	The pointer. Stable across rebuilds by design.

Everything else — gateway URL, Key Vault name, storage account, client IDs, cluster details — is read from the facts contract per run. A weekly rebuild therefore costs **zero reconciliation** across every fork and every mirror. That is the whole argument for discovery over push, and it is why the pointer is allowed to persist while the facts are not.

Four of those five values survive a full teardown and rebuild. That is not a happy accident; it is what Book Two's decision to leave managed identities standing through `spi down` was *for*.

The Borrow

The lane is one credentialed job, `deploy-test`, chained behind a gate job, `deploy-gate`, in the validation workflow. It runs on the fork's own pull requests and again on the push after merge, because a merge is a new build with a new digest.



The borrow transaction. The lane pins its own image into a running environment, proves it with the declared suites, and restores the canonical image whether or not the tests passed.


```

gate    → trusted event? onboarded? descriptor present? image pushed?
login   → Azure OIDC as the environment's deploy identity
install → the spi release the environment reports (environment.stackVersion)
gate    → spi status --json : is the environment deployable?
borrow  → spi service pin --image ghcr...@sha256:... --ephemeral --run-id $GITHUB_RUN_ID
verify  → spi service verify : poll until the live pod's imageID is our digest
mint    → three tokens: RESOLVER_TOKEN, RESOLVER_MEMBER_TOKEN, RESOLVER_NO_ACCESS_TOKEN
bind    → resolver: descriptor × facts × tokens → <suite>.env
prove   → docker run --env-file <suite>.env <svc>--acceptance@<digest> (per suite)
restore → spi service reset --if-run $GITHUB_RUN_ID (if: always)
verdict → Validation Summary

```

Several steps carry more design than their single line suggests.

The gate decides who may hold a credential

`deploy-gate` runs without credentials and decides whether the credentialed job may start at all. It permits only a `push` or a same-repository `pull_request`; it rejects Dependabot; it rejects `fork_upstream`, which in filter mode contains no Azure source and therefore produces no image to deploy. It then requires the onboarding values to be present, a descriptor to exist, an image to have actually been pushed, and the acceptance image not to have been skipped.

Each refusal is reported as a *named reason* in the summary rather than as a silent skip. When the pointer variables are absent, the gate says “repository is not onboarded to a stack,” the lane does not run, and the required check stays green. This is what lets a single organization-wide ruleset serve every fork, onboarded or not, without per-repository check configuration. The alternative — stripping the required check from forks that are not onboarded — was rejected because it hides an absence rather than explaining it.

The job then enters the GitHub environment named `spi-stack`, which is how the OIDC subject from the previous chapter comes to be asserted, and takes a concurrency slot:

```

concurrency:
  group: spi-stack-${vars.SERVICE_NAME || github.event.repository.name}
  cancel-in-progress: false

```

Per service, not per environment, and explicitly *not* cancelling. A borrow in progress must be allowed to finish and restore; cancelling it mid-transaction is precisely the thing that strands a pin.

The client matches the environment

Then a step that looks redundant and is quietly important.

The lane first installs the latest released `spi` wheel from the platform repository. It then runs `spi status --json`, reads `.environment.stackVersion`, and — if that value is a release tag of the form `vX.Y.Z` — **reinstalls that exact version**, replacing the one it just installed.

The client matches the server. Because the shared environment is pinned to a release tag rather than a branch, the environment can always say which version of the tooling it was deployed with, and the fork's CI can align itself to that rather than assume compatibility. If the environment reports a branch or has no recorded version, the lane keeps the latest release, which is the only sensible fallback.

This is a small piece of engineering with an outsized effect: it means the contract between the two repositories is versioned in practice and not merely in principle.

Deployability is asked, never remembered

Next the lane asks whether the environment will accept a deployment at all. `spi status --json` is polled every 20 seconds for up to 10 minutes. If `deployable` never becomes true, the step fails and reports the typed reason it was given — including `maintenance`, which is how Book Two's "a failed lifecycle run leaves the maintenance flag set" becomes visible three repositories away.

This is the replacement for the sticky boolean. Live status, every run, with a reason attached.

Deploy is a lock write, not a cluster mutation

Now the borrow itself, and the mechanism that makes the whole thing possible.

The naive approach — `kubectl set image` — does not work in a GitOps environment, because Flux reverts it. The obvious workaround, suspending Flux for the duration of CI, was rejected by the platform's own register: it costs self-healing for as long as it lasts, and it fails the very first deploy after every rebuild.

Two further alternatives were considered and rejected. Publishing a per-deploy Helm chart and re-pointing the `HelmRelease` source moves CI into the release pipeline. Suspending only the owning Kustomization freezes every sibling service in the same tree, which defeats the point of a shared environment.

What happens instead is a single write:

```
spi service pin "$SERVICE" --image "$SERVICE_IMAGE" \  
  --ephemeral --run-id "$GITHUB_RUN_ID" \  
  --source-repo "$GITHUB_REPOSITORY" --source-sha "$GITHUB_SHA" \  
  --source-run-url "$GITHUB_SERVER_URL/$GITHUB_REPOSITORY/actions/runs/$GITHUB_RUN_ID"
```

This is a compare-and-set update, guarded by `resourceVersion`, against the `osdu-image-lock` ConfigMap in `osdu-flux` — the same object Book Two introduced as the platform's ordinary image resolution mechanism. The pins live in a JSON annotation, `spi-stack.osdu.dev/pins`, that overlays

the canonical entries beneath; a companion annotation, `spi-stack.osdu.dev/trusted-repos`, projects the onboarding roster into the cluster. The object carries `reconcile.fluxcd.io/watch: Enabled`, so writing it triggers Flux to re-render the affected release from the new value.

GitOps stays alive throughout. Nothing is suspended, no manifest is patched, and the platform's normal reconciliation is the thing that performs the deploy. In the platform's own phrasing, *a lock write is the whole deploy*.

The record written is explicitly a loan rather than a change of state. It is marked `ephemeral: true`, it carries `origin: github`, and it records who to blame and where to look: `run_id`, `source_repo`, `source_sha`, and `source_run_url`. The image itself is pinned by digest — `repository@digest` — not by tag, because tags move and digests do not.

Refusal means two different things

The retry logic around the pin encodes a distinction worth copying.

If `spi service pin` refuses, the lane re-reads `spi status --json`. If the environment is *still deployable*, the refusal was not about readiness — something else went wrong, and the lane fails immediately rather than retrying into a wall. If the environment has *become* undeployable, the refusal is a scheduling problem, and the lane waits up to another ten minutes, polling every twenty seconds, before giving up.

The same command returning the same error is interpreted differently depending on what the environment says about itself. That is only possible because readiness is a separate, machine-readable contract rather than an inference from a failed call.

Proving It

The pin returns before the image is live

`spi service pin` writes a ConfigMap and returns. It does not wait for the new pod, and it *cannot* — the fork's identity deliberately holds no Flux write permission and no ability to create or delete anything. Reconciliation happens asynchronously, triggered by the watch label.

So the lane polls:

```
spi service verify "$SERVICE" --image "$SERVICE_IMAGE" --json
```

requiring `.outcome == "verified"`, every 20 seconds, for up to 15 minutes. Verification compares the *running pod's* `imageID` against the digest that was pushed — not the manifest, not the lock, the live container.

One outcome short-circuits the loop. A typed `lock_mismatch` means the lock no longer holds this run's pin: somebody else's borrow replaced it mid-transaction. Retrying cannot help, so the lane fails at once.

Secretless callers

No long-lived credential exists anywhere in this path.

GitHub's OIDC token is exchanged for the Azure federated identity established at onboarding, using a digest-pinned `azure/login` action. The Azure roles are the two minimal ones from the previous chapter, and the Kubernetes permissions are patch-on-one-named-ConfigMap plus read-only pods and logs.

For the tests themselves, three bearer tokens are minted per run and reach the suites under fixed names:

- **RESOLVER_TOKEN** — the privileged caller, minted for the audience the environment publishes as `.azure.token_audience`.
- **RESOLVER_MEMBER_TOKEN** — a plain user with no administrative rights.
- **RESOLVER_NO_ACCESS_TOKEN** — a caller with no entitlements at all.

The latter two are minted from the run's own OIDC assertion, and only if the environment actually publishes the corresponding client IDs in its facts. Their purpose is to make negative authorization tests writable: *this user should be refused* is an assertion that requires a user who will, in fact, be refused. Without a second and third identity, an acceptance suite can only ever prove that the powerful caller succeeds.

A developer mints the identical three with `spi token`, `spi token --member`, and `spi token --no-access`. That is what makes the local loop genuinely identical to CI rather than merely similar — and it is why the token command’s peculiar insistence on Entra’s v1 endpoint, from Book Two, matters here: a v2 token would project a different `x-app-id` and the tests would behave differently by the door.

Binding, then running

Proving happens in two passes. First the resolver is asked what suites exist, without resolving anything:

```
resolve.py --contract-only --descriptor .spi/service.yaml --report suites.json
```

Then, for each declared suite, the resolver joins the descriptor against the facts envelope and the minted tokens to produce an environment file, and the suite image is run against it:

```
resolve.py --mode run --suite <suite> --facts facts.json --env-file <suite>.env  
docker run --env-file <suite>.env -e SUITE_DIR=<dir> <svc>--acceptance@<digest> <mvn args>
```

each under the suite’s own declared `timeoutMinutes`.

A detail from the developer documentation is a small security point worth preserving: the generated `.env` file is `docker run data, never shell code`. Maven does not read it, and sourcing it in a shell would evaluate secret values as shell commands.

Tests as images

The acceptance suite is built as its own container image, `<svc>--acceptance:<sha>`, from the same commit as the service image, with Maven dependencies pre-warmed at build time.

The rationale is the provable-release constraint: running the tests that shipped with a given release, months later, must stay one command. Bare Maven on a runner cannot offer that, because it re-resolves dependencies from a moving repository every time.

The project is precise about the limit of this guarantee. The image pins the *suite source* and a warmed local repository; it does not pin dependency resolution. The run is online, and version ranges in the upstream dependency graph can still resolve differently later.

One deliberate judgment call: the suite image runs **on the GitHub runner**, executing `docker run` against the public gateway, rather than as a Job inside the cluster. In-cluster would be closer to production networking, but it requires create and delete verbs that the fork identity deliberately does not have — the security posture of chapter two decides the test topology of chapter five. The

accepted cost is that runner-to-gateway egress must remain open, which is true today and flagged for revisit if the environment ever moves behind a private endpoint.

Answers that must be created

Some answers cannot be discovered, because nothing creates them. A brand-new environment has no legal tag and no entitlements for a test caller, and a suite that needs either simply fails.

The design separates seeded state into three tiers by lifetime and ownership:

- **Tier 1, Bootstrap** — the baseline the platform is unusable without: legal tag, partition configuration, entitlements groups, schemas, and the test-identity step. This is Book Two’s bootstrap chapter, reconciled by GitOps on every rebuild, idempotent by HTTP status, with results published back as facts.
- **Tier 2, Named loads** — persistent shared data such as reference datasets. Labeled Kubernetes Jobs driven by a YAML registry, batch-loaded with duplicate skipping, run as the last stage of the rebuild pipeline rather than by hand.
- **Tier 3, Fixtures** — per-run, identity-shaped state, created at test start and torn down in a `finally` block, with orphans reaped by the next run.

The boundary rule is adopted verbatim from a predecessor project: *persistent state becomes a declared load; per-run identity becomes an ephemeral fixture*.

“Ready” and “seeded” are deliberately separate signals. A failed eighty-thousand-record data load should never block platform readiness — which is why, back in the status envelope, they are different fields.

Giving It Back

Restore is ownership-checked and unconditional

The restore step runs on `if: always()` — after passing tests, after failing tests, after a resolver error, after a timeout.

```
spi service reset "$SERVICE" --if-run "$GITHUB_RUN_ID" --json
```

`--if-run` is the important flag. It restores the recorded canonical image **only while the live pin still belongs to this run**. If the pin has been replaced by a newer borrow, or was already cleaned up, the command exits `2` and the lane treats that as success — because in both cases the correct outcome has already been achieved by somebody else, and overwriting it would be worse than doing nothing. Exit `1` is a genuine restore failure and fails the lane.

This is the difference between “undo my change” and “set the image back to what I remember it being.” Only the first is safe when several actors share one environment.

Best effort, with a backstop

The project is candid that this is best-effort. A runner that dies outright — cancelled, evicted, or lost — never executes its restore step, and the pin is stranded. The environment keeps running a fork’s candidate image with no test in progress and nobody watching.

Two safety nets exist. Pins are ephemeral records, not configuration, so the intent is recoverable from the annotation itself: every pin names its run and the URL of that run. And a sweep runs on weekdays to clear ephemeral pins whose owning run is no longer active, using an age threshold of three hours so that a legitimately slow borrow is never swept out from under itself.

A cluster-wide deploy lock was considered as an alternative and rejected: it would serialize every service’s CI behind every other service’s, converting a shared environment into a queue. A cluster-side `Lease` for pin admission was rejected on the same grounds.

What sharing actually costs

Because concurrency is per service rather than per environment, sibling services borrow simultaneously by design. The platform’s register is unusually honest about what that means, and these are real hazards rather than theoretical ones:

A sibling service can **restart during another service’s test run**, because a pin write triggers reconciliation of that service’s release, and a neighbor’s suite may be mid-request when its dependency’s pod cycles.

Worse, a fork image that is **behaviorally broken but passes readiness** will be considered successfully deployed, and can break the acceptance suites of every concurrent sibling that depends on it — producing a red result in a repository whose code is fine.

The accepted costs are stated plainly elsewhere too: a busy fork queues rather than cancels, and a different service holding the environment can produce up to ten minutes of deployability polling before a borrow begins. Each hold is capped by the suite's declared timeout.

The verdict

Finally, `validation-summary` always runs and emits the required check. Its design rule is that verdicts carry attribution: environment-not-ready, infrastructure error, and test failure are distinct outcomes, reported distinctly. A suite that collects zero tests is never a pass. An interrupted run is never a pass.

One honest wrinkle comes with the design that lets one ruleset serve every fork: *a skipped deploy lane can leave that summary green*. A repository that is not onboarded reports a green Validation Summary having never deployed anything, which is correct — it genuinely has nothing to report — but it means a green check is a statement about the checks that ran, not a promise that the deploy lane ran.

What the Seam Reveals

One vocabulary, two repositories

Step back from the mechanism and something structural is visible.

The same idea appears on both sides of this seam, invented separately. Book One's filter refuses an unknown path pattern rather than guessing; the resolver refuses an unknown source kind rather than guessing. The platform's status command returns a typed `reason.code` rather than a boolean; the resolver returns exit codes 2, 3, and 4 rather than failing generically. The platform separates `ready` from `deployable` from `seeded`; the lane separates `environment-not-ready` from `infrastructure error` from `test failure`.

Neither repository imported this from the other. Both arrived at it because both are operated by people who have been woken up by an ambiguous failure. The engineering system states the principle in one line, quoted in Book One: *guessing would have been convenient exactly once, and wrong repeatedly*.

The handshake works because that principle is shared. A typed refusal on one side is legible as a typed refusal on the other, and every decision point in this book — may I borrow, is it live, did the suite run, may I restore — is answered with a reason rather than a boolean.

What is not yet wired

Both projects publish what they have not built. Collected here, with the fork side first:

- **Key Vault materialization is not wired.** Descriptors may declare `keyvault:` bindings and the resolver validates them; the lane does not yet pass a secrets file.
- **Declared requirements are not enforced before borrowing.** A descriptor's `requires.loads`, `requires.groups`, and `dependencies` are validated as syntax but not compared against published environment state, so a suite can still borrow an environment lacking the data it declared. The intended behavior is a typed refusal — *environment not seeded: reference-data* — instead of the industry-standard failure mode of an acceptance test returning zero results and a human reading a runbook.
- **A second image verification immediately before each suite is not implemented.** Verification runs after the pin, not again at suite start — so a sibling's pin landing in between is detected late.
- **The drift tripwire is pending.** The design calls for a build-stage static check, needing no cluster and no secrets, that fails the build when suite code reads an environment variable the descriptor does not declare, printing the file, the line, and a paste-ready declaration. The point is that a new requirement surfaces at review time rather than as a red deployment three repositories downstream.

And on the platform side:

- **spi service refresh** is explicitly unbuilt.
- **The source-policy promotion machinery is ahead of the code.** The `--canonical-source` flag and the `spi-source-<service>` tags are designed and documented; the refresh workflow's backstop step and declaration enforcement are not yet implemented.
- **Scheduled reset and teardown workflows remain unbuilt**, which is why the weekly cadence described in Book Two is currently an operator practice rather than an automation.

One further caution belongs here because it is easy to misread as a gap and is not. The platform's smoke workflow tears down with `az group delete --no-wait` followed by `|| true`, which means *a green teardown job does not prove deletion*. It proves the request was accepted.

Which document governs

Both projects maintain design documents alongside decision records, and the design documents describe intended capability. The engineering system adds an unusual instruction about its own authority: where the deploy-and-test design document and the two decision records disagree, **the records govern**. The design describes intended capabilities and is explicitly not a checklist of implemented gates.

That instruction is the right note on which to end this book. The handshake described here is real, shipped, and running — the borrow, the proof, and the restore all work today. Around it sits a larger design, partly built, openly tracked, and documented in enough detail that anyone can tell which is which.

That is rarer than it should be, and it is the strongest argument that the two systems were built by people who expected to be held to what they wrote down.

Epilogue

The Things That Are Not True

A collection of statements that sound correct, are frequently assumed, and are wrong. Each is a real misconception that one of the two projects takes the trouble to correct in its own documentation.

About the engineering system

“fork_upstream is a mirror of upstream.” Not at the first tier. It is a *generated* tree: upstream’s shared code with non-Azure providers and deployment machinery removed and Azure module references injected. It has never contained the fork’s Azure source, and it is not a faithful copy of anything. Only at the customer mirror tier is it verbatim, and that is what the `SYNC_MODE` variable selects.

“The daily sync merges upstream into the fork.” Nothing is merged. The tree is computed from the upstream tip and written directly with Git plumbing. The resulting commit is *merge-shaped* — two parents, upstream reachable — but that shape is a record of provenance, not the output of a merge algorithm. This is the entire reason the recurring modify/delete conflict cannot occur.

“The filter removes the other providers from the repository’s history.” It removes them from the generated tree. Everything that was ever committed remains in history and remains reachable. The filter controls what the branch contains, not what the repository remembers.

“A green check on the sync pull request means the fork builds.” It means the *provider-less* tree builds, with a Maven profile of `core` alone. There is no Azure source on `fork_upstream` to compile. Whether the fork actually builds is established later, in the cascade, on `fork_integration`.

“-P azure builds the core and Azure profiles.” It builds Azure only. Maven deactivates every `activeByDefault` profile the moment any explicit `-P` is passed, so `core` silently drops out and module resolution fails somewhere that looks unrelated. The profile is always written as `core,azure`.

“The cascade runs automatically after a sync.” It is designed to be triggered by a person, after they review the sync pull request. `Cascade Monitor` sweeps every six hours to catch cascades that were forgotten or stuck, but the documentation is explicit that the sweeper is a safety net behind the manual path and not a replacement for it.

“Removing a label is housekeeping.” Removing `human-required` is a control signal. It is what tells the monitor that a human has finished fixing the problem and the cascade may be retried. Labels here are the state machine, not decoration.

“Upstream bug fixes to the Azure provider will eventually arrive.” They will not. The Azure provider and test trees were seeded once and are fork-owned from that moment. Synchronization never writes to them again. A late upstream fix to Azure source requires somebody to port it deliberately.

“A release rebuilds the container image.” It retags one. Validation already built and pushed an image identified by its commit SHA; the release adds a semantic-version tag to that same image. The bits that were tested are the bits that ship, because they are the same bits.

“A passing CodeQL check means CodeQL analyzed the code.” Not necessarily. CodeQL is the name of a summary job that always runs and reports the aggregate outcome. On a documentation-only or configuration-only pull request, analysis is correctly skipped and the summary still reports success. This is deliberate — it exists precisely so those pull requests are not blocked forever — but the check’s name promises more than its presence delivers.

“Coverage is enforced.” JaCoCo runs and reports. No threshold gates anything.

“External contributors get the full pipeline.” They get the build. They do not get an image push and they do not get a deployment signal, because credential-bearing jobs are excluded from external heads by design. A maintainer-authorized override is described in the schema and explicitly not wired.

“If the tooling can detect a problem, it can fix it.” Settings Apply checks whether the container registry package is public and reports when it is not. It cannot correct it, because no API exists to change package visibility. Detection and remediation are separate capabilities, and this system has one without the other in at least one place.

“The AI-generated pull request descriptions were AI-generated.” They were not. For the entire life of the reference fork, the tool crashed on every invocation and the deterministic fallback template produced every description that was ever published — while the workflow reported success and reviewers merged without noticing. That story is the subject of the next chapter.

About the platform

“spi up finished, so OSDU is ready.” No. A successful CLI exit is not an API-readiness check. Flux is still reconciling, operators are still starting middleware, and the schema loader may have well over an hour of work remaining.

“ready means every Kustomization is ready.” Explicitly not. ready is its own computed verdict, and deployable is a third thing again — an environment can be perfectly healthy and still refuse deployments because it is in maintenance, has no deploy record, or has a pending entitlements bootstrap.

“Suspending the Git source stops Flux.” It stops Flux *fetching new commits*. The controllers keep reconciling the cached revision, so in-cluster drift is still corrected the whole time the source is suspended.

“`spi reconcile` will pick up my Git changes.” Not while the source is suspended. Plain `spi reconcile` does not temporarily resume a suspended source; `spi reconcile --resume` does. An operator who edits Git, reconciles, and waits for a change that never arrives has met this behavior.

“A Kustomization reporting Ready means the service works.” A `Certificate` whose status is never updated can pass a Flux health check, producing a green Kustomization while HTTPS never opens. Readiness of an object is not readiness of a service.

“A failing HelmRelease will keep retrying.” It will not. A release that exhausts `install.remediation.retries` is marked `Stalled=True` with reason `RetriesExceeded`, and helm-controller stops retrying it. It will sit there, failed and quiet, until somebody intervenes.

“A 404 from the gateway means ingress is broken.” It means the opposite of broken. A completed HTTPS request returning 404 has already passed DNS resolution, TCP connection, the TLS handshake, and gateway routing. It is the best failure on the list of six.

“The bare profile is cheaper.” Profiles select Kubernetes workloads, not a smaller Azure estate. `spi up` provisions AKS and the full set of PaaS templates for every profile, `bare` included. It saves reconciliation time, not money.

“Keys are still available as a fallback.” They are not. `disableLocalAuth: true` is hardcoded on Cosmos and Service Bus, and Storage has shared-key access disabled. There is no fallback path, which is why community images that build a connection string fail permanently rather than degrading.

“`spi down` deletes the environment.” It deletes the resources in the group, individually, and deliberately leaves the managed identities, the resource group, and its tags behind so that onboarded repositories keep working. `spi down --purge` is the real deletion.

“A green teardown job means the resources were deleted.” The smoke workflow tears down with `az group delete --no-wait` followed by `|| true`. A green result proves the request was accepted, not that anything was removed.

“An ARM deployment either succeeds or rolls back.” *An ARM deployment is not a transaction.* A failure leaves some resources created and some not; recovery is to fix the template and run it again, which works only because the templates are written to be idempotent.

About the seam

“A green Validation Summary means the change was deployed and tested against a live environment.” Only if the fork is onboarded to one. If the pointer variables are absent, the deploy

gate reports that the repository is not onboarded, the lane does not run, and the summary stays green. The visible skip is intentional, so that one ruleset can serve every fork; but green does not by itself mean proven.

“A descriptor’s declared requirements gate the lane.” Not yet. `requires.loads`, `requires.groups`, and `dependencies` are validated as syntax but are not compared against the environment’s published state before borrowing. A suite can still run against an environment missing the data it declared.

“Onboarding a repository makes it the source of the environment’s image.” It does not. Onboarding grants trust — permission to borrow. Which image the environment runs by default is a separate, explicitly recorded decision, and reverting it does not revoke trust.

“Restore puts the image back.” Restore puts the image back *if this run still owns the pin*. If a newer borrow has replaced it, restore deliberately does nothing and reports success, because overwriting a live sibling’s borrow would be worse than leaving it alone.

Writing Down What Was Learned

Between them the two projects maintain seventy-two architectural decision records — thirty-eight in the engineering system and thirty-four in the platform. Every trade-off described in this book is drawn from them, which is the only reason a document like this can claim to report the projects' reasoning rather than an outsider's reconstruction of it.

The engineering system goes one step further and keeps a second document, and the relationship between the two is unusual enough to be worth describing.

A register organized by question

The register is organized by *the question each record answers* rather than by chronology, and each record carries a stated impact level.

The numbering runs from 001 to 041 with three numbers — 017, 032, and 034 — retired and deliberately left unused. That detail is a small piece of intellectual honesty: rather than renumbering to stay contiguous and thereby invalidating every existing citation, the corpus leaves the gaps and says why they are there.

The records cover fork management, synchronization and cascade, release automation, template propagation, workflow patterns, and most recently continuous integration and deployment. Each preserves not merely what was chosen but what was rejected, with the rejected option's genuine advantage stated in a line. The platform's register has the same character; the catalogue of six rejected image-management strategies in Book Two, and the four rejected authorization designs in its identity chapter, are its work rather than this book's.

A second document, derived rather than authoritative

The second document is a file of learnings — short essays, one lesson each, written in the first person plural, each ending in a pointer to the decision record that holds the detail.

The separation is the interesting part. The register states what is true. The learnings state what was *understood*, and they are explicitly derived rather than authoritative: when the two could disagree, the record governs. This frees the learnings to be narrative, to describe a failure in the order it actually unfolded, and to generalize — things a decision record is a poor container for.

The headings alone carry most of the argument, and several have become the operating principles of the system:

Isolation is what lets the team keep shipping. Split what fails differently. Labels are the state machine. State that must survive a runner lives in GitHub metadata. Add metadata instead of rewriting history. Know the event context before building a workaround. Enforce trust boundaries

*in the workflow, not in review. Generate the branch; do not merge into it. Halt on the unknown.
No environment values in the repository.*

The most useful entry is about a failure

One learning is worth reproducing at length, because it is the clearest example of the corpus's value and of the habit that produces it.

The system once generated pull request descriptions with an AI model, designed to degrade gracefully to a structured template when the model was unavailable. The design was reasonable. The graceful degradation worked exactly as specified.

The outcome, recorded plainly: the tool **crashed on every single invocation for the entire life of the reference fork**. The fallback was not a fallback; it was the only path that ever ran. The workflow reported success throughout. Reviewers read the template-generated descriptions and merged them without complaint. Nobody noticed for months.

The lesson the project draws inverts the original intent:

A fallback that cannot be distinguished from success is not resilience, it is a blindfold.

The response was not to fix the integration. Descriptions became deterministic, and the commit classification that drives versioning became a fixed rule rather than a judgment. The register also records why the feature was not simply replaced: the model platform it used was retired, the coding-agent API rejects the App installation tokens this system authenticates with, and the remaining option has no API surface at all. Reopening the question requires a provider clearing all three constraints, and none currently does.

Three things make that entry unusually good. It names a date and a pull request. It states an outcome that reflects poorly on an earlier decision by the same authors. And it generalizes to a rule that applies well beyond the feature that produced it.

It is also, read against this book as a whole, the seed of everything in it. A blindfold is what a sticky `DEPLOY_VALIDATED` flag was. It is what a fallback to a community image would be when a fork image is missing. It is what a green teardown that deletes nothing is, and what a `Certificate` passing a health check without opening HTTPS is. The same failure shape recurs across both repositories, and in each case the fix is the same: make the difference between working and not working visible in the output.

The habit behind it

Read together, the documents show a consistent posture that explains most of what this book has described.

The recurring instinct is to **refuse to proceed on an assumption**. The filter halts on an unclassified path rather than defaulting. The resolver halts on an unknown source kind. The synchronization state manager fails the workflow on an API read error rather than concluding that no pull request exists. The deploy gate consults live status every run rather than trusting a stored flag. The platform resolves availability zones before creating anything rather than discovering the problem halfway through, and refuses to fall back to a community image when a fork image cannot be resolved.

The second instinct is to **put the enforcement where it cannot drift**. The trust boundary is a guard clause in the workflow, not a convention reviewers are expected to notice. Branch protection is a checked-in ruleset reconciled weekly, not a settings page someone configured once. Pod security compliance is baked into a locally owned Helm chart rather than patched onto somebody else's at deploy time. The Dockerfile is owned centrally, because the distributed version had been broken for a long time without anyone knowing.

The third is to **make state durable and observable**. A process spanning days across stateless runners keeps its state in labels, issue markers, and repository variables, in the open, where a human can read it and a workflow can query it. An environment's source policy is a resource-group tag that survives the cluster it describes. An ephemeral pin names the run that created it and the URL where that run can be read.

It is worth considering how much of this book's usefulness comes from the fact that someone wrote those lessons down while they still stung.

Glossary

Adopt Fork The workflow that onboards a customer-tier repository: it detects the parent as upstream, creates the branch topology, and sets `SYNC_MODE=mirror`.

AKS Automatic An opinionated Azure Kubernetes Service mode in which Azure operates node provisioning, the service mesh, and an admission policy layer. Requires Kubernetes 1.36 or above for the platform's webhook-dependent operators to install.

Bootstrap The one-time population of an empty OSDU environment: partition records, entitlements root groups, a legal tag, entitlements membership, and schemas. Rendered as idempotent one-shot Jobs and reconciled on every rebuild.

Borrow, prove, restore The deployment lane's transaction: pin a freshly built image into a running environment, run the declared acceptance suites against it, and restore the canonical image unconditionally.

Canonical source The durable record of where a service's image normally comes from — `community` or a named fork — stored as a resource-group tag. Distinct from trust, and never inferred from it.

Cascade The integration phase. After a synchronization pull request merges into `fork_upstream`, the cascade merges `main` into `fork_integration`, merges `fork_upstream` on top, validates, and offers the result to `main` as a release pull request.

Cascade monitor A workflow running every six hours that detects missed, stuck, or repaired cascades and resumes them. A safety net behind the manual path, not a replacement for it.

Correlation tag A Git tag formatted `<release-tag>-upstream-<upstream-version>`, recording which upstream release a fork release contains.

Deployable The environment's verdict on whether it will currently accept a deployment. Separate from `ready`: a healthy environment may be undeployable because it is in maintenance, has no deploy record, or has an incomplete bootstrap.

Deploy identity A managed identity, one per environment, federated to a specific fork and GitHub environment. Holds least-privilege Azure roles and namespace-scoped Kubernetes permissions. Survives teardown so that onboarded repositories keep working.

Deploy record The environment's record of what was deployed and when, written to resource-group tags and a ConfigMap, and reported in the status envelope.

Descriptor `.spi/service.yaml`. The fork-owned declaration of what a service's test suites need, expressed as symbolic bindings rather than values. Excluded from synchronization and protected by CODEOWNERS.

Entitlements The OSDU service answering authorization questions, backed on Azure by a Cosmos DB Gremlin graph.

Ephemeral pin A run-owned, digest-pinned image override written into the image lock, marked `ephemeral` and carrying the run that created it. Restored by that run, and swept if the run dies.

Facts The environment's machine-readable self-description, published by the Stack's CLI as `spi info --json` and `spi status --json` under `apiVersion: spi.osdu.dev/v1`. Read per run, never cached.

Filter-Rev A commit trailer identifying the filter that produced a generated tree: the engine version joined to a hash of the configuration file. The sentinel value `mirror` marks the customer tier.

Flux The GitOps engine reconciling Kubernetes manifests from Git, installed as a native AKS extension. Continues reconciling a cached revision while its source is suspended.

fork_integration The integration workspace branch. Where upstream and fork-owned code are combined, conflicts resolved, versions stamped, and validation run. Allowed to break; reset to `main` after each release.

fork_upstream The upstream-derived branch. At the first tier it is a generated, filtered tree with no Azure source. At the mirror tier it is a verbatim copy of the parent's `main`. Written only by synchronization, read only by the cascade.

Gateway The Kubernetes Gateway API object through which traffic enters. Bound to the managed Istio add-on's existing ingress Service rather than creating one.

Generated branch A branch whose content is computed from inputs rather than produced by merging. The central mechanism of the engineering system.

GitOps The practice of treating a Git repository as the source of truth for cluster state, with a controller continuously reconciling the difference.

Halt The filter's and the resolver's response to an input they cannot classify: stop with exit code 2 and open a labeled issue naming what was not understood. Never guess.

HelmRelease A Flux custom resource declaring a Helm chart installation. Every OSDU service is one, against a single locally owned chart.

human-required The label meaning a person must act before automation can continue. Its *removal* is the signal that permits a retry.

Image lock The `osdu-image-lock` ConfigMap recording which image each service runs, substituted into manifests by Flux. The deployment lane writes to it under compare-and-set. A lock write is the whole deploy.

Kustomization A Flux object reconciling a directory of manifests, declaring dependencies on other Kustomizations and a readiness wait.

Maintenance An environment flag that blocks deployments without asserting the environment is unhealthy. Set deliberately by a failed lifecycle run so that a human looks before CI proceeds.

Meta commit One synthetic conventional commit added per synchronization, summarizing the upstream range, whose type is derived by a fixed rule so that version computation stays deterministic without rewriting upstream history.

Mirror mode Customer-tier synchronization behavior, selected by `SYNC_MODE=mirror`, taking the upstream tip verbatim through the same commit plumbing while the filter engine never runs.

Onboarding Granting a fork repository permission to borrow an environment, by adding a federated credential to the deploy identity for the subject `repo:<org>/<fork>:environment:spi-stack`. Grants trust only, never source policy.

OSDU The Open Subsurface Data Universe: an open standard and reference implementation for energy industry data, developed under the OSDU Forum.

Partition An OSDU tenant boundary. On Azure each partition receives its own Cosmos DB account, Service Bus namespace, and Storage account.

Profile Which Kubernetes workloads an environment reconciles — `bare`, `minimal`, or `core`. Selects workloads, not the size of the Azure estate.

Provider model OSDU's structure separating shared service logic from swappable cloud-specific implementations, conventionally as `<svc>-core` alongside `provider/<svc>-azure` and siblings.

ReferenceGrant A Gateway API object permitting a resource in one namespace to be referenced from another. Used so a Gateway can read a TLS Secret cert-manager was only allowed to write elsewhere.

Ruleset Branch protection expressed as checked-in JSON and reconciled onto the repository weekly, rather than configured by hand.

Safeguards AKS Automatic's admission layer, enforcing pod security requirements. The reason the platform owns a local Helm chart rather than patching upstream ones.

Seeding The one-time population of the fork-owned Azure provider and test trees at initialization, taken from the newest upstream revision that still contains them.

Service provider interface (SPI) OSDU's pluggability seam between shared logic and cloud-specific implementation. Also, confusingly, the name of the two Azure projects built around it.

SPI Stack `osdu-spi-stack`, the deployment platform of Book Two. Consumer of the engineering system's images and lender of the environment its lane borrows.

Suite A named group of tests declared in a descriptor, with its own bindings and timeout, packaged in the acceptance image.

Suspension Setting `spec.suspend` on the Flux Git source so that new commits are not fetched. Does not stop reconciliation of the already-cached revision.

SYNC_MODE The repository variable selecting the fork tier. Unset means filter mode; `mirror` means the customer tier. The tier cannot be inferred from the tree, which is why the variable exists.

Template repository `osdu-spi` itself. It owns the workflow logic; each fork owns its configuration data.

Template sync The daily workflow that opens a pull request in each fork when the template has moved, so one fix propagates everywhere without hand edits.

Tier Which kind of fork a repository is. The first tier is a Microsoft-maintained service repository filtered from the community project; the customer tier is a true GitHub fork mirroring a service repository.

Tracking issue An issue used as durable workflow state and as the audit trail for a process spanning hours or days. Carries the hidden marker recording the active upstream SHA.

trust-manager The controller mirroring certificate authority bundles across namespaces, so services in one namespace can trust middleware in another.

Trust boundary The rule that credential-bearing jobs run only where the code being built could not have been supplied by an untrusted party. Enforced as a guard clause on each sensitive job.

Upstream-Sha A commit trailer recording the exact upstream commit a generated tree was derived from.

Version stamping The cascade's update of fork-owned Maven POM versions so they stay consistent with upstream-derived coordinates that now move independently.

Workload Identity The mechanism by which a Kubernetes pod proves its identity to Entra ID using a projected ServiceAccount token, removing the need for any stored Azure credential.

Appendix A: The Engineering System Register

Thirty-eight standing records from `Azure/osdu-spi`, each stated as its ruling. Numbers 017, 032, and 034 are retired and intentionally unused.

#	Title	Ruling
001	Three-Branch Fork Management	Three long-lived branches — <code>main</code> , <code>fork_upstream</code> , <code>fork_integration</code> — so conflict resolution happens away from production.
002	GitHub Actions Automation	GitHub Actions is the automation substrate; the system is designed around GitHub event semantics.
003	Template Repository Pattern	Forks self-configure from a template that owns the logic while each fork owns its data.
004	Release Please Versioning	Versions and changelogs are computed from conventional commits rather than maintained by hand.
005	Automated Conflict Management	Conflicts are isolated and resolved in the integration branch; upstream never lands on <code>main</code> directly.
006	Two-Workflow Initialization	Initialization splits into two workflows so user input and repository setup fail separately.
007	Initialization Bootstrap	Logic required in a fork's first commit lives in local actions, outside the path that would deliver its own fix.
008	Centralized Label Management	Labels are defined in one file and are part of the system contract, not per-workflow decoration.
009	Asymmetric Cascade Review	Review scrutiny differs by stage; symmetric review on every hop was rejected as slower and noisier.
010	YAML-Safe Shell Scripting	Shell logic is written to be YAML-safe, which pushes substantial logic out of workflow files.
011	Configuration-Driven Template Sync	Template synchronization is driven by configuration files rather than per-fork hardcoding.

#	Title	Ruling
012	Template Update Propagation	Template changes reach forks as reviewable pull requests; one fix propagates everywhere.
013	Reusable Actions for PR Creation	Pull request creation is factored into reusable composite actions instead of duplicated inline.
014	AI-Enhanced Development Workflow	Retired. The AI path crashed on every invocation; descriptions are now deterministic and classification is a rule.
015	Template-Workflows Separation	<code>.github/workflows/</code> is template-only; <code>.github/template-workflows/</code> is what forks receive.
016	Initialization Security Handling	Initialization treats credentials and permissions explicitly rather than as content copying.
018	Fork-Resources Staging	Fork-specific resources are staged before deployment rather than applied in a single step.
019	Cascade Monitor	A six-hourly sweeper detects missed cascades, retries repaired ones, and escalates stale conflicts.
020	Human-Required Label	<code>human-required</code> is the manual-intervention signal; assignment to people failed on orgs, renames, and API errors.
021	Pull Request Target Trigger	Cascade triggers use <code>pull_request_target</code> so the workflow runs from the target branch, never the PR branch.
022	Issue Lifecycle Tracking	Tracking issues hold workflow state and serve as the audit trail for multi-day processes.
023	Meta Commit Strategy	One synthetic commit per sync, typed by a fixed rule; upstream history is never rewritten.
024	Sync Duplicate Prevention	Sync state is durable, label-driven, and fails closed; exactly one pull request per upstream state.
025	Java/Maven Build Architecture	Java 17, Maven, and JaCoCo through reusable actions, against the community Maven repository.
026	Dependabot Strategy	Split by ownership: the template scans its own dependencies; forks scan Maven only, patch-level only.

#	Title	Ruling
027	Documentation Generation	Documentation is Markdown built by MkDocs Material and published from the template only.
028	Workflow Script Extraction	Substantial bash moves into scripts wrapped by composite actions, organized by lifecycle.
029	GitHub App Authentication	All elevated automation uses GitHub App installation tokens; personal access tokens are excluded.
030	CodeQL Summary Job	A job named exactly <code>CodeQL</code> always reports, and the <code>code_scanning</code> ruleset requirement is removed.
031	Template Sync Duplicate Prevention	Template sync reuses its existing labeled branch and pull request rather than opening a new one daily.
033	GHCR as Service Image Registry	Public GHCR is the registry; pulls are anonymous, and a visibility flip is a known unfixable hazard.
035	Azure-Only Maven Profile	CI builds <code>core,azure</code> by default with a per-repository override; a bare <code>-P</code> is never emitted.
036	Workflow Trust Boundaries	Credential-bearing jobs run only in trusted contexts; the guard clause is the policy, not review.
037	Engineering System Owns the Dockerfile	One canonical <code>build/Dockerfile</code> packages the Maven-built JAR; service-owned Dockerfiles are unused.
038	Upstream Filter and Azure Seeding	<code>fork_upstream</code> is generated by a filter, not merged; Azure source is seeded once and fork-owned forever.
039	Customer-Tier Mirror Sync	A second tier mirrors the finished product verbatim, selected by a variable because the tree cannot say.
040	Descriptor Acceptance Contract	Each fork owns a declarative descriptor; the template owns the resolver that joins it to runtime facts.
041	Borrow, Prove, Restore Lane	One credentialed job borrows the environment for its image, proves every declared suite, and always restores.

Appendix B: The Platform Register

Thirty-four records from `Azure/osdu-spi-stack`, each stated as its ruling.

#	Title	Ruling
001	Azure PaaS for Data Services	Every data service with a managed Azure equivalent uses it; a different cloud is a fork, not a flag.
002	AKS Automatic as Compute Substrate	Azure operates the cluster; Kubernetes 1.36 is a floor because mutating webhooks are blocked below it.
003	In-Cluster Middleware Scope	Exactly three stateful systems stay in-cluster: Elasticsearch, Redis, and Airflow's PostgreSQL.
004	Local Helm Chart for Safeguards Compliance	One locally owned chart bakes pod security in, rather than patching upstream charts at deploy time.
005	Workload Identity for Azure PaaS Access	One managed identity federated to a ServiceAccount; no Azure credential is stored in the cluster.
006	Three-Namespace Model	Create only <code>foundation</code> , <code>platform</code> , and <code>osdu</code> ; the managed mesh owns its own namespaces.
007	Layered Flux Kustomization Ordering	Workloads reconcile as explicitly ordered layers, because apply order is not a dependency graph.
008	Bicep for Azure Provisioning	Azure resources are hand-written Bicep; the CLI owns only what templates cannot express.
009	Flux CD + AKS GitOps Extension	Flux as a native AKS extension, with <code>stack</code> and <code>ingress</code> as separate top-level Kustomizations.
010	Key Vault + ConfigMap Secret Model	Secrets are split by class; Azure PaaS credentials never land in Kubernetes.
011	Cross-Namespace CA Distribution	Two trust-manager Bundles mirror the Redis and Elasticsearch CAs into the OSDU namespace.
012	Three Ingress Profiles	<code>azure</code> , <code>dns</code> , and <code>ip</code> are three complete trees rather than one mode with conditionals.

#	Title	Ruling
013	Schema Load via a Flux-Managed Job	A Job, not a sleeping Deployment, because a Deployment that only loads data lies to Kubernetes.
014	Suspend GitOps After Deploy	<code>spi up</code> suspends the Git source so a development cluster stops moving underneath an investigation.
015	Partition, Entitlements, and Legal Bootstrap	Four idempotent one-shot Jobs per partition, rendered by a single Flux-managed chart.
016	Istio JWT Projection	The mesh strips and rewrites identity headers, because the provider chain reads headers, not principals.
017	Per-Deploy Image Lock	Image references live in one ConfigMap substituted by Flux; pins overlay it without touching Git.
018	Karpenter NodePool Authoring	Capacity is application code in Git, reviewed as custom resources, not declared in Bicep.
019	SPI-Owned GitOps Objects in <code>osdu-flux</code>	The platform's own GitOps objects live in a namespace policy allows it to write.
020	Opt-In Application Insights	Provisioned only on request; the bundled agent is kept inert rather than half-configured.
021	Middleware-Only <code>minimal</code> Profile	<code>bare</code> , <code>minimal</code> , <code>core</code> ; the unbacked <code>full</code> profile was deleted rather than documented as missing.
022	TLS Certificates in <code>platform</code>	Certificates live where cert-manager may write; a ReferenceGrant lets the Gateway read across.
023	Entra-Only Data Plane	<code>disableLocalAuth</code> is hardcoded on Cosmos and Service Bus; there is no key-based fallback.
024	Windows Batch Shim Launcher	Every subprocess goes through one explicitly escaped <code>cmd.exe</code> command line.
025	One Flux Inventory Owner per Object	Exactly one Kustomization owns each object; shared ownership caused mutual pruning.
026	Bind to the AKS Managed Istio Ingress	The Gateway binds to the add-on's existing ingress Service rather than deploying another.

#	Title	Ruling
027	Subscription-Resolved Availability Zones	Usable zones are resolved from the subscription before any resource is created.
028	Version-Pinned Shared Environment	The shared environment deploys a release tag, never a branch, declared in one reviewed file.
029	Environment Lifecycle and Reset Boundary	status , refresh , upgrade , reset , teardown ; only a full reset clears data state.
030	Machine-Readable Status and Deploy Record	A versioned status envelope with typed reasons and exit codes; facts are a separate command.
031	Fork Images Deploy as Ephemeral Pins	A fork deploy is a run-owned ephemeral pin with an always-run restore and a sweep backstop.
032	Environment Deploy Identity and RBAC	One deploy identity per environment, federated per repository, with two narrow namespace Roles.
033	Explicit Canonical Image Source Policy	Trust and canonical source are separate durable records; there is no silent fallback.
034	Managed Identities Survive spi down	Teardown leaves identities standing so onboarded repositories keep working; purge is the real deletion.

Appendix C: Operational Reference

Fork workflows

Run only in the template `osdu-spi`, never deployed to a fork:

File	Name	Purpose
<code>init.yml</code>	Initialize Fork	Detects a new fork, creates labels, opens the initialization issue
<code>init-complete.yml</code>	Initialize Complete	Reads the issue reply and configures the fork end to end
<code>dev-ci.yml</code>	Template Development CI	The template's own build and test harness
<code>dev-release.yml</code>	Release Management	The template's own release automation
<code>docs.yml</code>	Deploy Documentation	Builds and publishes the documentation site
<code>codeql.yml</code>	CodeQL Analysis	Security analysis of the template
<code>scorecard.yml</code>	OpenSSF Scorecard	Supply-chain posture scoring

Delivered from `.github/template-workflows/` at initialization and kept current by template sync:

File	Name	Trigger
<code>sync.yml</code>	Sync Upstream	Daily, and manual dispatch
<code>cascade.yml</code>	Cascade Integration	Manual dispatch, and from the monitor
<code>cascade-monitor.yml</code>	Cascade Monitor	Every six hours, on sync PR close, and manual
<code>validate.yml</code>	Validation	Pull requests and pushes to the three branches
<code>build.yml</code>	Build and Test	Pushes to ordinary feature branches only
<code>release.yml</code>	Release Management	Pushes to <code>main</code> , excluding <code>.github</code> -only changes
<code>integration-cleanup.yml</code>	Integration Branch Cleanup	A merged release pull request on <code>main</code>
<code>sync-template.yml</code>	Sync Template	Daily
<code>adopt-fork.yml</code>	Adopt Fork	Manual dispatch, customer tier
<code>settings-apply.yml</code>	Settings Apply	Weekly, and manual
<code>dependabot-validation.yml</code>	Dependabot Validation	Dependabot pull requests

File	Name	Trigger
<code>ghcr-retention.yml</code>	GHCR Retention	Weekly
<code>codeql.yml</code>	CodeQL Analysis	Weekly and on code changes
<code>copilot-setup-steps.yml</code>	Copilot Setup Steps	Agent environment preparation

Schedules

Cron (UTC)	What runs
<code>0 0 * * *</code>	Upstream synchronization
<code>0 8 * * *</code>	Template synchronization
<code>0 */6 * * *</code>	Cascade monitor
<code>0 4 * * 1</code>	Settings apply
<code>0 5 * * 1</code>	Registry retention
<code>0 6 * * 1</code>	CodeQL analysis
<code>30 1 * * 0</code>	OpenSSF Scorecard

Labels

Defined centrally in `.github/labels.json` and applied consistently by every workflow.

Lifecycle: `initialization`, `upstream-sync`, `sync-update`, `sync-failed`, `template-sync`, `template-update`, `template-sync-failed`, `release-tracking`, `production-ready`

Cascade state: `cascade-active`, `cascade-blocked`, `cascade-ready`, `cascade-failed`, `cascade-escalated`, `upstream-held`, `integration-blocked`, `validated`, `validation-failed`

Intervention: `human-required`, `needs-resolution`, `conflict`, `manual-review-required`, `escalation`, `high-priority`, `emergency`, `rollback`

Other: `build-failed`, `dependencies`, `copilot`

Platform commands

Command	What it does
<code>spi check</code>	Verify prerequisites before provisioning
<code>spi up --env <env></code>	Provision Azure, bootstrap the cluster, activate GitOps, suspend the source

Command	What it does
<code>spi up --dry-run</code>	Render an <code>az deployment group what-if</code> diff without creating anything
<code>spi down --env <env></code>	Delete environment resources, retaining identities, the group, and its tags
<code>spi down --purge</code>	Remove external grants, then delete the group and the identities
<code>spi status / --watch / --json</code>	Report readiness, deployability, maintenance, and a typed reason
<code>spi info --json</code>	Publish endpoints, partitions, and non-secret Azure coordinates
<code>spi connect</code>	Point <code>kubectl</code> at the environment's cluster
<code>spi reconcile</code>	Reconcile from the cached revision
<code>spi reconcile --resume / --suspend</code>	Clear or set suspension on the Git source
<code>spi reconcile --refresh-images</code>	Move canonical image entries in the lock
<code>spi onboard <service> --repo <org>/<fork></code>	Grant a repository trust to borrow this environment
<code>spi onboard --canonical-source fork\ community</code>	Set which source supplies a service's default image
<code>spi service pin <service> --image <ref> --ephemeral</code>	Write a run-owned image pin into the lock
<code>spi service verify <service> --image <ref></code>	Confirm the running pod carries the requested digest
<code>spi service reset <service> --if-run <id></code>	Restore the canonical image if this run still owns the pin
<code>spi token / --member / --no-access</code>	Mint the same three test callers CI uses
<code>spi maintenance</code>	Set or clear the flag that blocks deployments

Sources

Everything in this document derives from two repositories.

Azure/osdu-spi — the engineering system of Book One:

- `README.md`, `CONTRIBUTING.md`, and `CLAUDE.md`
- `doc/src/index.md` and `doc/src/concepts.md`
- `doc/src/architecture/` — the overview, the three-branch strategy, the workflow system, fork tiers, and the deploy and test design
- `doc/src/workflows/` — adoption, initialization, synchronization, cascade, validation, build, and release
- `doc/src/runbooks/` — the fork lifecycle and the service descriptor
- `doc/src/adr/` — the thirty-eight-record decision register, its index, and the companion learnings file
- `.github/` — the workflows, template workflows, composite actions, local actions, rulesets, and label definitions

Azure/osdu-spi-stack — the platform of Book Two:

- `README.md` and `CHANGELOG.md`
- `docs/architecture.md` and `docs/install.md`
- `docs/design/` — the Bicep architecture, workload identity, fork deployment, gateway ingress, deployment lifecycle, Flux reconciliation, secret lifecycle, environment lifecycle, and CI smoke guides
- `docs/decisions/` — the thirty-four-record register
- `infra/` — the Bicep entry points and modules
- `software/` — the charts, components, and profile stacks reconciled by Flux
- `src/spi/` — the CLI, including its status, pins, images, onboarding, and guard modules

Both repositories are licensed under Apache 2.0.

The handshake of Book Three was reconstructed by reading both sides against each other — the fork’s validation lane and acceptance resolver, and the platform’s status contract, image lock, and onboarding code — and every exact string, exit code, poll interval, and file path cited here was read from source rather than from prose about it.